

澳門理工大學

Universidade Politécnica de Macau

Macao Polytechnic University

Faculty of Applied Sciences

A Sample Thesis — Pattern Matching Based on Object Graphs — for Using the MPU LaTeX Thesis Template

–

*A **Thesis** submitted for the Examination*

in partial fulfillment of the requirements for the degree of

Doctor of Philosophy in Computer Applied Technology, Macao Polytechnic University

May 2026


0
cm
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24



澳門理工大學
Universidade Politécnica de Macau
Macao Polytechnic University

Thesis

A Sample Thesis — Pattern Matching Based on Object Graphs — for Using the MPU LaTeX Thesis Template

Name	–
Student Number	–
Academic Unit	Faculty of Applied Sciences
Programme	Doctor of Philosophy in Computer Applied Technology
Supervisor	–
Date	6th May 2026


0
cm
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24

DECLARATION

I hereby declare that this Thesis titled **A Sample Thesis — Pattern Matching Based on Object Graphs — for Using the MPU LaTeX Thesis Template** has been composed solely by myself, that the work contained herein is my own except where explicitly stated otherwise in the text, and that this work has not previously been submitted in part or in full for this degree or any other degree.

I declare that I am aware of and understand the Macao Polytechnic University (MPU) Guidelines for Plagiarism Avoidance, Rules Regarding Cheating and Other Violations of Examination Regulations, and rules and regulations related to the academic integrity of students.

–

Faculty of Applied Sciences
Macao Polytechnic University


0
cm
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24

ABSTRACT

Pattern matching has been widely adopted in functional programming languages, and is gradually getting popular in OO languages, from Scala to Python. The structural pattern matching currently in use has its foundation on algebraic data types from functional languages. To better reflect the pointer structures of OO programs, we propose a pattern matching extension to general statically typed OO languages based on object graphs. By this extension, we support patterns having aliasing and circular referencing, that are typically found in pointer structures. With the requirement of only an abstract subtyping preorder on types, our extension is not restricted to a particular hierarchical class model. We give the formal base of the graph model, that is able to handle aliases and cycles in patterns, together with the abstract syntax to construct the object graphs. More complex cases of conjunction and disjunction of multiple patterns are explored with resolution. We present the type checking rules and operational semantics to reason about the soundness by proving the type safety. We also discuss the design decisions, applicability and limitation of our pattern matching extension.

Keywords — pattern matching, object graph, subtyping, type system, operational semantics, programming language


0
cm
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24

论文样例 — 基于对象图的模式匹配 — MPU LaTeX 论文模板用例

摘要

模式匹配在函数式语言中已得到广泛应用，并逐渐在面向对象语言中普及，如 Scala 与 Python。现有的结构化模式匹配源于函数式语言的代数数据类型。为更好地刻画面向对象程序中的指针结构，本文提出一种基于对象图的通用静态类型面向对象语言的模式匹配扩展。该扩展支持包含别名与循环引用的模式，这些特性在指针结构中常见。由于仅依赖于类型上的抽象子类型预序，该方法不受限于特定的类层次模型。本文形式化给出了能够处理别名与循环的图模型，并提供了构造对象图的抽象语法。同时，我们探讨了多模式合取与析取的复杂情形及其解析机制。进一步地，本文提出了类型检查规则与操作语义，并通过证明类型安全性来论证其健全性。最后，我们讨论了该模式匹配扩展的设计取舍、适用范围及其局限性。

关键字 — 模式匹配、对象图、子类型、类型系统、操作语义、程序设计语言


0
cm
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24

論文樣例 — 基于對象圖的模式匹配 — MPU LaTeX 論文模板用例

摘要

模式匹配在函數式語言中已得到廣泛應用，並逐漸在面向對象語言中普及，如 Scala 與 Python。 現有的結構化模式匹配源于函數式語言的代數數據類型。 更好地刻畫面向對象程序中的指針結構，本文提出一種基于對象圖的通用靜態類型面向對象語言的模式匹配擴展。 該擴展支持包含別名與循環引用的模式，這些特性在指針結構中常見。由于僅依賴于類型上的抽象子類型預序，該方法不受限于特定的類層次模型。 本文形式化給出了能處理別名與循環的圖模型，並提供了構造對象圖的抽象語法。 同時，我們探討了多模式合取與析取的複雜情形及其解析機制。 進一步地，本文提出了類型檢查規則與操作語義，並通過證明類型安全性來論證其健全性。 最後，我們討論了該模式匹配擴展的設計取捨、適用範圍及其局限性。

關鍵字 — 模式匹配、 對象圖、 子類型、 類型系統、 操作語義、 程序設計語言

ACKNOWLEDGEMENTS

TABLE OF CONTENTS

Declaration i

Abstract iii

Acknowledgements ix

Table of Contents xi

List of Figures xiii

List of Tables xv

List of Algorithms xvii

List of Abbreviations xix

Chapter 1. Introduction 1

1.1 Algebraic Pattern Matching . . . 1

1.2 Graph-based Pattern Matching . . . 3

1.3 Contribution and Outline . . . 4

1.3.1 Current Publication . . . 5

1.3.2 Thesis Outline . . . 5

Chapter 2. Related Work 7

2.1 Pattern Matching in OOP . . . 7

2.2 Graph-Based Formalism . . . 8

2.3 Graph Pattern Matching . . . 9

Chapter 3. Pattern Graphs and Match Relation 11

3.1 Layout Graphs . . . 11

3.2 Match Relation . . . 12

3.3 Pattern Graphs . . . 15

3.4 Match Conjunction and Disjunction . . . 17

Chapter 4. Pattern Language 23

4.1 Syntax . . . 23

4.2 Pattern Graph Construction . . . 24

4.3 Type-Checking . . . 25

4.4 Operational Semantics . . . 26

4.5 Type Safety 27

Chapter 5. Discussion 31

Chapter 6. Conclusion 33

Chapter 7. About This Template 35

7.1 Document Class and Options 35
7.2 Commands for Setup 36
7.3 Commands for Display 40
7.4 Emphasis of Author Names in Bibliography 41
7.5 Double-blind Review Format 42
7.6 Preloaded Class and Packages 43
7.7 History 43

Bibliography 51

Appendix A. Implementation Source Code 55

A.1 Abstract syntax of all else 55
A.2 Abstract syntax of pattern matching 55
A.3 Prety-printing of dictionaries 56
A.4 Bijective dictionary 57
A.5 Graphs and graph algorithms 58
A.6 Operations on values 63
A.7 State transition rules 65
A.8 Types and subtyping rules 69
A.9 Type-checking rules 73
A.10 Test cases 78

Chapter B. GNU General Public License Text 87

Chapter C. Algorithms as Inference Rules 101

Chapter D. Biography 105

LIST OF FIGURES

1.1 Syntax tree of an algebraic pattern 2
1.2 Object with aliased and circular relations 2
1.3 Junction of multiple patterns 3

3.1 Subgraph isomorphism and covering between layout graphs 13

LIST OF TABLES

3.1 Subtyping Rules 16

4.1 Abstract Syntax of Pattern Matching 24
4.2 Type-Checking Rules 29
4.3 State Transition Rules 30

LIST OF ALGORITHMS

3.1 Construction of Match Morphism 15
3.2 Construction of Union 18
3.3 Construction of Intersection 20
3.4 Construction of Match Conjunction 21
3.5 Construction of Match Disjunction 22

4.1 Construction of Pattern Graph 28

LIST OF ABBREVIATIONS

E	the set of edges in a graph
G	a graph
N	the set of nodes in a graph
$\mathscr{C}$	the alphabet of classes
$\mathscr{A}$	the alphabet of graph edge labels
$\mathscr{N}$	the alphabet of graph nodes
$\mathscr{V}$	the alphabet of values
`@`	"`as`" operator to bind a new variable to a structure
$\curlywedge$	graph conjunction
$\curlyvee$	graph disjunction
$\sqcap$	non-deterministic choice
$\gtrsim_f$	covering, under graph isomorphism f
$\gtrsim\!\!\rightrightarrows$	covering de-aliased graph of
$\rightrightarrows_f$	de-aliased from, under total function f
$\sqsubset_f$	matching, under graph isomorphism f
$\multimap$	subclass of, is-a
$\cong_f$	isomorphic to, under bijective function f
$\lesssim\!\!\rightrightarrows$	covered by de-aliased graph of
$\xrightarrow{a}$	an edge with label a
$\rightarrow^*$	a path, possibly empty, leading to
$\odot$	re-rooting operation
$\prec$	subtype of, preorder
C++	C++ programming language
DFS	depth-first search
OO	object oriented
RDF	resource description framework
SPARQL	SPARQL protocol and RDF query language
MPU	Macao Polytechnic University


0
cm
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24

Chapter 1.

INTRODUCTION

Contents

1.1 Algebraic Pattern Matching 1

1.2 Graph-based Pattern Matching 3

1.3 Contribution and Outline 4

1.3.1 Current Publication 5

1.3.2 Thesis Outline 5

Pattern matching is a programming language mechanism to test a structure against a pattern at run-time. It has two main purposes, matching and decomposition. Matching is to determine whether the tested structure has the particular components organized in the particular way specified in the pattern. Once the matching is successful, the individual components of the structure can be accessed through new variable bindings introduced in the pattern. Pattern matching supersedes conditional statements in many occasions by stronger specificity, better integrity and higher abstraction, where conditions are organized as a whole structure that is more intuitive and efficient, easier to comprehend, and less complicated.

1.1 Algebraic Pattern Matching

Pattern matching is already common in functional programming. Patterns in the functional world follow the scheme of algebraic data types, or variants, where data objects are generated by hierarchical constructor applications. The patterns are therefore *partial* applications, structured as abstract syntax trees of function invocations. Figure 1.1 shows the syntax tree of a typical binary search tree pattern in a functional language, where x, y and z provide new bindings of variables to the nodes in the application tree. Such a pattern follows the hierarchically structured algebraic constructors, that is simple and easy to match with very clear semantics. That's why pattern matching is widely used in the very early ages of functional languages.

When pattern matching is migrated to OO languages, subtyping between classes must be brought into consideration. A great benefit gained is the ability to perform type tests with variable bindings in an atomic way, that is much safer and eliminates the need of typecasts. However, the algebraic nature of patterns does not change very much. An additional layer is commonly introduced to bridge the difference between the algebraic

$$Branch(Branch(x, 2, y \mathbin{@} Branch(Leaf, 3, Leaf)), 4, z)$$

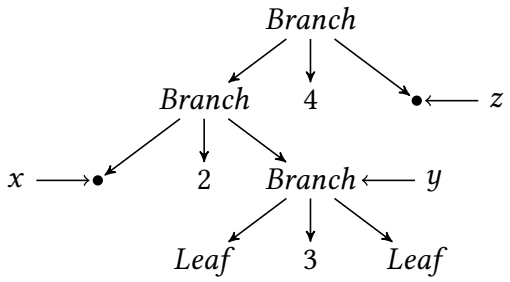


Figure 1.1: Syntax tree of an algebraic pattern

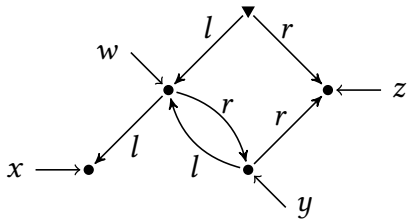


Figure 1.2: Object with aliased and circular relations

view [1] and the underlying pointer structure of an object, for example, the *case class* in Scala [2]. Although the case class constructors create objects with stored structures identical to normal objects, those stored structures are in fact irrelevant. The Scala *extractors* can completely virtualize the case class constructors and abstract the stored structures away. Such pattern matching matches an object against it abstract meaning expressed by the algebraic view rather than the underlying stored structure. Therefore, the semantics of patterns is still in the functional world, with the benefit of higher level abstraction, and the limitation of unable to examine the stored relations of data objects.

The decision to put OO pattern matching in a functional way has its reasons. Besides that the semantics of algebraic patterns is simpler and more familiar to programmers who already know functional pattern matching, the underlying pointer structures of objects can have aliases and cycles, that are more complicated than trees. Figure 1.2 shows an object with aliased and circular relations. Furthermore, when considering the conjunction and disjunction of multiple patterns with subtyping, a type that is compatible with many types must be resolved. Nevertheless, there's a need to expose and explore the concrete stored structures of objects when writing internal implementations for OO methods. This is often the case where pattern matching is used, since diverging processing of variants externally in OO paradigm usually goes through the subtyping polymorphism. Also, expressing the relations between objects explicitly in the stored structures is a key characteristic of OO programs. Therefore, by addressing these challenges, we propose a pattern matching extension to general OO languages that directly operates on the pointer structures of objects commonly found in most language imple-

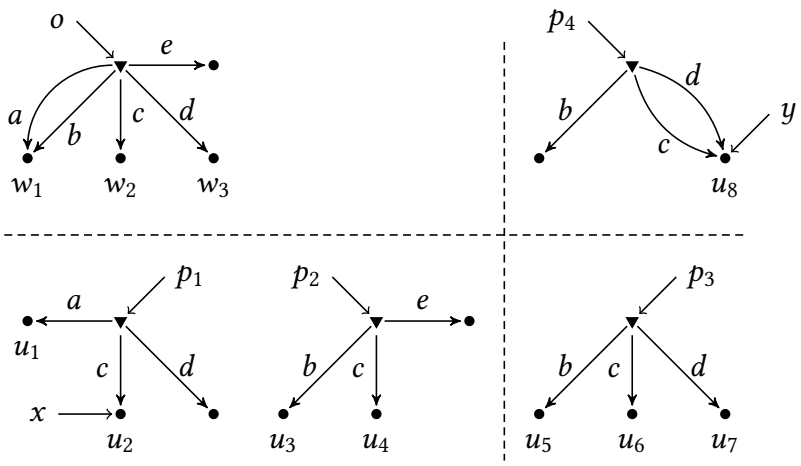


Figure 1.3: Junction of multiple patterns

mentations.

1.2 Graph-based Pattern Matching

We base the theoretical foundation of our extension on the well established object graph model [3]. An object in this model is a rooted and directed graph with labeled edges. The root represents the object with outgoing edges pointing to components. The edges are labeled with the attributes of the source node. Local variables mark the edges from the stack to roots of objects. Figure 1.3 shows an object graph on the upper-left, where local variable o leads to the object, which has attributes $o.a$, $o.b$, $o.c$, $o.d$ and $o.e$. Among the attributes, $o.a$ and $o.b$ alias each other and point to the same object. We omit the stack in the shown graphs, since it's somewhat irrelevant to pattern matching. Those edges without a source node are coming from the stack. In our pattern matching graph model, both objects and patterns are represented by the object graphs, which are collectively called pattern graphs in the context. The major difference is that a pattern is not a complete object, some or all of the attributes can be omitted.

Given the notion of pattern graphs, our pattern matching is intuitive. A pattern graph matches another when the one *covers* the other. To define the covering between two graphs, we need to figure out the correspondence between the nodes of the two graphs. Beginning with the roots, the nodes led to by the edges with the same label correspond to each other. Cumulatively, the label sequence from the root to a node is the incoming path, or the *trace*, of a node. Although a single path is sufficient to find a node, we must take into account the whole set of incoming paths to a node to obtain the graph structure. The idea to identify a node by how it can be reached is called the trace model [4], where an isomorphism can be built between a node and the set of its incoming paths. The trace model is crucially concise in reasoning about equivalent nodes in different graphs, and plays extremely well with aliases and cycles. There are

two aspects in determination of the covering — the layout test and the type test. First, a node w_1 covers another node u_1 when the incoming paths of w_1 include the incoming paths of u_1. To separate the case of aliased nodes, a node can only cover one node in the other graph. As shown in Figure 1.3, w_1 covers u_1 for $\{[a], [b]\} \supset \{[a]\}$, on the contrary, u_8 covers either u_6 or u_7, but not both. Second, the type $\tau(w_1)$ must be a subtype of $\tau(u_1)$. In OO languages, subtyping is a preorder, i.e., reflexive and transitive [5, 6]. We only stick with this abstraction in order to make the pattern matching general. When all the nodes of a pattern graph are covered by the nodes of the other, we say this pattern is matched by the other. In terms of subtyping, the match relation is covariant.

It's getting more interesting when we look further into the match against multiple patterns. One object can be tested to match all of the patterns, the conjunction, or at least one of the patterns, the disjunction. For example, in Figure 1.3, object o matches all of p_1, p_2 and p_3, and one of p_3 and p_4. While the tests of multiple patterns are straightforward, the bindings of variable x in the conjunction, and y in the disjunction are more subtle. For conjunction, if there exists a node w_2 that can cover u_2, u_4 and u_6, $\tau(w_2)$ must be a subtype of all $\tau(u_2)$, $\tau(u_4)$ and $\tau(u_6)$, therefore it's possible to assign a better type to variable x than just $\tau(u_2)$. Also, there exist invalid conjunctions which no graph can possibly match. In Figure 1.3, p_3 and p_4 cannot be matched by one graph. We need to detect and reject invalid conjunctions. For disjunction, we don't have conflicts between the multiple patterns. However, without knowing which pattern will be eventually matched at run-time, variable bindings are more complicated. For example, what is the type of variable y in the disjunction of p_3 and p_4, $\tau(u_8)$, $\tau(u_6)$ or $\tau(u_7)$? If object o matches the disjunction, for it matches p_3, which node do we bind to y, w_2 or w_3?

1.3 Contribution and Outline

With the brief discussion of the problems and ideas, below lists the challenges we are going to address in this work.

1. We build a formal base for the pattern graphs with subtyping. This includes the definitions, morphisms, operations, match relations and algorithms for matching. In addition, we design a language to construct the pattern graphs, in particular with aliases and cycles, as an extension to existing OO languages.

2. We study the compatibility of the pattern matching with the current type-checking and run-time environment, and find a way for smooth integration. We present the type-checking rules, operational semantics and reason about the type safety.

3. We explore the conjunction and disjunction of multiple patterns, and give our resolution, design decision and algorithms to handle the relatively complex cases at compile-time.

1.3.1 Current Publication

1.3.2 Thesis Outline

The rest of the thesis is outlined as follows. Chapter 2 summarizes the work in object graphs and pattern matching, from the perspective of OO programming. Chapter 3 defines the formal base of pattern matching on object graphs, including pattern conjunction and disjunction, with justifications and constructive algorithms. Chapter 4 presents the language extension for the graph-based pattern matching in abstract syntax, and provides the type-checking rules and operational semantics, together with the proof of type safety. Chapter 5 discusses the restriction and limitation of the theory, and the potential solutions. Finally, Chapter 6 concludes the work.


0
cm
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24

Chapter 2.

RELATED WORK

Contents

2.1 Pattern Matching in OOP 7
2.2 Graph-Based Formalism 8
2.3 Graph Pattern Matching 9

2.1 Pattern Matching in Object Oriented Programming

Besides Scala, there have been many attempts to promote pattern matching in OO programs. Because the complexity and multi-paradigm trend of OO languages, a one-for-all solution is difficult. These attempts each have their own focuses. Geller *et al.*, in a full-length technical report [7], proposed a pattern matching framework in a purely OO style for a dynamic typed language: Newspeak, one of the Smalltalk family. Patterns are like normal objects that receive messages to perform the matching operations. This proposal focuses on providing a framework for pattern messaging and pattern combinators, such as conjunction, disjunction, sequencing and negation, as well as the building blocks for structural matching, such as value literals and attribute keywords. How to match a pattern is left abstract and subject to individual object implementations. Widemann *et al.* [8] similarly provided a framework in a library approach requiring generics, that focused on the interfaces and protocols to match patterns against, extract components from and binding variables to objects. Conjunction and disjunction are also supported through the "both" and "either" combinators. Again, how objects and components match patterns is still abstract and therefore shallow to an object.

Solodkyy *et al.* [9] proposed an efficient functional-style pattern matching for C++ as a library, making heavy use of C++ *concepts* and template metaprogramming. A particular pattern is expressed as a template instantiation, and the properties specific to that pattern are constructed at compile-time. Plus the smart use of virtual table pointers as type identifiers for hashing, this gives very minimal performance overhead to the extension. Although C++ extensively supports pointers and references, but the structures are of value semantics. This library is not specific to pointer structures. Ryu *et al.* [10] provided an language extension to Fortress, an experimental language with OO perspective. The extension depends on the *getters* of an object to match its components, and focuses on run-time type tests and multimethods. The fashion of multimethods [11] is a strong

application of pattern matching in functional languages, and it has been extensively studied in OO languages in the contrast to the single-dispatch dominance [12, 13, 14].

Even a bold experiment was carried out in early days by Liu *et al.* [15] in their JMatch extension to Java. It generalizes functions and operations to be able to find the arguments back from the results, by introducing the forward and *backward* modes, including automatic iteration to search for elements in a collection. This is more aggressive than the explicit extractors in Scala to decompose objects, with some level of operation inference. Emir *et al.* [16] gave a thorough review of the commonly adopted OO pattern matching techniques, including object decomposition, visitors, type tests, typecase, case classes and extractors. All these techniques are currently available in Scala. Kohn *et al.* [17] proposed a complete solution of pattern matching to the dynamically typed Python language. It lives with the existing Python so well that it will greatly simplify the style of future Python code. This extension has been included in the official Python 3.10 [18, 19], and is thus far the most powerful algebraic pattern matching implementation in the OO world. Finally, OO pattern matching is widely accepted, not only in Scala and Python, it will appear in Java 16 [20] and C# 8 [21], the two most popular production OO languages.

2.2 Graph-Based Formalism

While most research on OO pattern matching all has focused on type tests and object decomposition in various abstract ways that coexist with the OO paradigm, with purposes to enhance type safety, expressiveness and conciseness, the pointer structures of the data models in OO programs are also of interest. The semantics of a program must eventually relies on how the data objects are represented [22]. Bornat [23] showed three example proofs of pointer programs in Hoare Logic, with treatment of pointer aliasing by the principle of *spatial separation*. Although the mechanism remains rather low-level, it achieved a level of local reasoning on particular data structures. Ke *et al.* [3] gave a small-step operational semantics of OO programs based on class, object and state graphs. It presents a formal graph model for full featured OO constructs, including stacks and method frames. Ke *et al.* [24] further extended the graph model to a generic type system, to handle recursive types, type variables and type instantiations in the notions of graph morphisms and transformations. Zhao *et al.* [25] employed the transformation of class and object graphs in the refinement of OO data structures, investigating what changes of patterns in the class structure maintained the capability of providing functionalities or services. Zhao *et al.* [26] proposed a graph-based Hoare Logic for reasoning about OO programs. The Hoare proof system consists of a set of logic rules covering most OO constructs, including object creation, local variable declaration and recursive method invocation. The soundness of the logic is given, that every specification proved by the system is valid. There wasn't a proof of completeness, although the logic is believed

complete.

2.3 Graph Pattern Matching

Besides the formulation of classes and objects, graphs are widely used in the abstraction of other language notions involving relations, such as query languages. Francis *et al.* [27] described the Cypher 9 query language of the Neo4j graph database. Cypher's core data model consists of values, property graphs and tables. The path patterns in the query language is quite similar to the objects in OO languages, where the nodes are entity patterns like types, and the edges are relations between entities like attributes. Formal semantics is given for the core query language based on the pattern matching. Tong *et al.* [28] proposed an approach for mapping OO database models into the resource description framework (RDF). This work gives the formal definitions of OO databases and RDF, and maps the query language of OODB to the SPARQL query of RDF based on graph patterns. Li *et al.* [29] defined taxonomy graphs and simulation admits the "is-a" relation similar to the OO subtyping, but on edge labels rather than node types. By this "is-a" relation, pattern matching on edges is relaxed such that a descendant edge matches an ancestor edge, and the relaxation is bounded by an inheritance distance. Although the purpose of the relaxation was to capture more sensible matches in real-life complex data graphs, the idea can be applied to abstract relations between objects in OO programs.

Chapter 3.

PATTERN GRAPHS AND MATCH RELATION

Contents

3.1 Layout Graphs 11
3.2 Match Relation 12
3.3 Pattern Graphs 15
3.4 Match Conjunction and Disjunction 17

We first give the formal definitions of graphs, operations, morphisms and relations. We begin with the basic layout graphs consisting of only nodes and edges, and the match relation w.r.t. to a given preorder. Then, we add type assignments to the layout graphs to form the pattern graphs, and describe how to encode program states as graphs. We further expand the theory to match conjunction and disjunction, discuss the restriction and provide our resolution.

3.1 Layout Graphs

Let $\mathscr{N}$ be the set of all nodes, which can be any unique entities, usually associated with types, values and objects, and $\mathscr{A}$ the set of all labels, which are usually variable names. We define the layout graphs on these elements.

Definition 1 (Layout Graph) A layout graph $G = \langle N, E, p \rangle$ is an edge-labeled, rooted and directed graph, where

1. $\text{nodes}(G) = N \subset \mathscr{N}$ is the set of nodes,
2. $\text{edges}(G) = E : N \to \mathscr{A} \to N$ is a partial function, in infix notation, representing the edges, mapping the source node and the label to the target node, and
3. $\text{root}(G) = p \in N$ is the root, and all the nodes are connected to p, $\forall u \in N \bullet \exists \overline{a} \bullet p\overline{Ea} = u$.[1]

[1] The overline $\overline{a}$ notation represents a series $a_1, \ldots, a_n$ of items. However, an variable E bound in the context, if overlined, repeats itself rather than expands into a series, thus $\overline{Ea}$ expands to $Ea_1Ea_2 \ldots Ea_n$.

We formulate the edges in a layout graph as a *curried* function to imply that the outgoing edges from a node must have unique labels. We write, more intuitively, $u \xrightarrow{a} v \in E$ to denote an edge $uEa = v$. Also, $u \rightarrow^* v \in E$ denotes there exists a path from u to v, including single-node paths.

A layout graph represents the layout of an object, where the root becomes the origin. Other nodes of a layout graph are the origins of object components, and each component is represented by the subgraph connecting to its origin.

Definition 2 (Re-rooting) Let $G = \langle N, E, p \rangle$ be a layout graph, and $p' \in N$. We say $G' = \langle N', E', p' \rangle$ is the subgraph re-rooted at node p', denoted as $G' = G \odot p'$, where

1. the nodes are connected to p',

$$N' = \{v \mid \exists p' \rightarrow^* v \in E\},$$

2. edges are restricted to the connected nodes, $E' = E|_{N'}$.

Because there may be circular paths going back to the root, re-rooting cannot guarantee a *proper* subgraph. Therefore, it does not necessarily imply a proper reduction. We must explicitly remove all the incoming edges to the root, and examine the residue subgraph. We define the subtraction to remove some edges from a layout graph.

Definition 3 (Subtraction) Let $G = \langle N, E, p \rangle$ be a layout graph, and $X \subseteq E$ the edges to exclude. $G' = \langle N', E', p \rangle$ is the subtraction graph of X from G, denoted as $G' = G \setminus X$, where

1. the nodes are connected via $I = E \setminus X$ to the root,

$$N' = \{v \mid \exists p \rightarrow^* v \in I\},$$

2. edges are restricted to the connected nodes, $E' = I|_{N'}$.

Since morphisms are functions on nodes, the subtraction gives us a way to break cycles in a graph for a proper reduction on the number of nodes through re-rooting.

3.2 Match Relation

The structural equivalence of two layout graphs is formalized as an isomorphism, which is a bijection preserving all the outgoing edges of a node.

Definition 4 (Graph Isomorphism) Let $G = \langle N, E, p \rangle$ and $G' = \langle N', E', p' \rangle$ be two layout graphs. G is isomorphic to G', denoted as $G \cong_f G'$, if there exists a bijective function $f : N \rightarrow N'$, such that

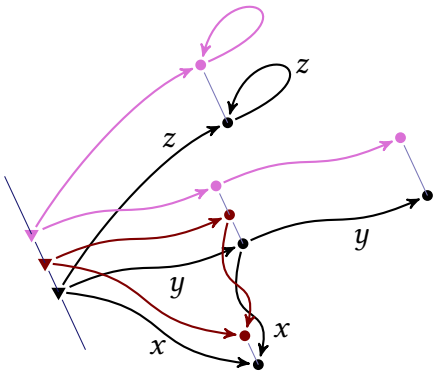


Figure 3.1: Subgraph isomorphism and covering between layout graphs

1. root maps to root, $f(p) = p'$, and
2. edges are preserved, $u \xrightarrow{a} v \in E \iff f(u) \xrightarrow{a} f(v) \in E'$.

If G is isomorphic to a root-preserving subgraph G'' of G', i.e., $\text{root}(G'') = \text{root}(G')$, we say G' *covers* G, denoted as $G' \gtrsim_f G$ or $G \lesssim_f G'$.

Figure 3.1 illustrates the covering between layout graphs. An object not only has the layout, but also has a type. We associate the type of an object to the root of its layout graph through a type-assignment τ. Suppose there is a preorder $\prec$ defined on these types, e.g., the subclass or subset relation. The type-assignment is there because nodes of graphs are unique, it's inconvenient to reason about ordering between nodes directly. We define the match relation $\sqsubset$ between two layout graphs.

Definition 5 (Graph Match) Let G and G' be two layout graphs, and $\prec$ a preorder on $\text{ran}(\tau)$. G' matches G w.r.t. $\prec$ under isomorphism f, denoted as $G' \sqsubset_f|_\prec G$, or simply $G' \sqsubset_f G$ without ambiguity, if $G' \gtrsim_f G$, such that the preorder exists between the types of any pair of corresponding nodes, i.e., $\forall u \in N \bullet \tau(f(u)) \prec \tau(u)$.

The graph match is a cover relation restricted to the given preorder between the types of nodes. It is trivial to verify that the graph match $\sqsubset$ relation is reflexive. We can also prove that $\sqsubset$ is transitive.

Theorem 1 The graph match $\sqsubset$ relation is a preorder.

Proof. Suppose $G' = \langle N', E', p' \rangle \sqsubset_f G = \langle N, E, p \rangle$ and $G'' = \langle N'', E'', p'' \rangle \sqsubset_{f'} G'$. We can verify

1. $f'(f(p)) = p''$,
2. $u \xrightarrow{a} v \in E \implies f'(f(u)) \xrightarrow{a} f'(f(v)) \in E''$, and
3. $\forall u \in N \bullet \tau(f'(f(u))) \prec \tau(u)$.

Hence $G'' \sqsubset_{f' \circ f} G$. The $\sqsubset$ relation is transitive. □

Obviously, if a layout graph G is matched by another graph G', any re-rooted subgraph of G must also be matched by the subgraph of G' re-rooted at the corresponding node.

Theorem 2 If $G' \sqsubset_f G = \langle N, E, p \rangle$, $q \in N$ and $X \subseteq E$, then $G' \odot f(q) \sqsubset (G \setminus X) \odot q$.

Proof. By restricting f to the nodes of $(G \setminus X) \odot q$. □

Corollary 1 If there are two morphisms f_1 and f_2 such that $G' \sqsubset G$ under f_1 and f_2, then $f_1 = f_2$, i.e., the match morphism is unique.

Proof. By induction on the number of nodes. We remove all the incoming edges to the root, and apply Theorem 2. □

On the other hand, by first removing all the incoming edges to the root, we can then re-root the residue subgraph to the adjacent nodes of the root, to reduce the match problem to the subgraphs. By combining the morphisms for these re-rooted subgraphs, if they exist, and adding back the root to the combined morphism, we are able to obtain the morphism for the original layout graph, if it exists.

Theorem 3 Given two layout graphs $G = \langle N, E, p \rangle$ and $G' = \langle N', E', p' \rangle$, Let $G^* = G \setminus \{u \xrightarrow{a} v \in E \mid v = p\}$ be the residue subgraph of G with all the incoming edges to the root removed. If

1. for all the outgoing edges from the root $p \xrightarrow{a} q \in E$,

$$\exists q' \bullet p' \xrightarrow{a} q' \in E' \wedge (p = q \vee G' \odot q' \sqsubset_{f_q} G^* \odot q),$$

2. and for all the incoming edges to the root $u \xrightarrow{b} p \in E$,

$$\exists q \bullet f_q(u) \xrightarrow{b} p' \in E',$$

then $G' \sqsubset_f G$, where $f = \left(\bigcup_q f_q\right) \cup \{p \mapsto p'\}$.

Proof. Obviously, by the given conditions, the addition of $\{p \mapsto p'\}$ preserves the incoming and outgoing edges of the root. By the definition of layout graphs, all the nodes are connected to the root, therefore any node in a layout graph must be either the root, or in one of the subgraphs re-rooted at the nodes adjacent to the root. Hence all we need to prove is that $\bigcup_q f_q$ exists, i.e., if $u \in \mathrm{dom}(f_q) \cap \mathrm{dom}(f_r)$ then $f_q(u) = f_r(u)$. This is true by Theorem 2 and Corollary 1, that any (sub)match morphism is unique. □

Although finding the subgraph isomorphism is NP-complete in general, it can be efficiently done for rooted graphs by a simple depth-first search (DFS), led to by Theorem 3. We can determine whether there is a match between two layout graphs by trying to

Algorithm 3.1: Construction of Match Morphism

Input: preorder: $\prec$, type-assignment: τ, layout graphs: G and G'.
Output: match morphism: f if $G' \sqsubset_f G$.
Exception: *Mismatch*.

```
1  begin
2      ⟨N, E, p⟩ = G, ⟨N′, E′, p′⟩ = G′
3      f ← ∅
4      dfs_match(E, p, E′, p′)
5  end
6  function dfs_match(E, p, E′, p′) begin
7      if p ∈ dom(f) then
8          raise Mismatch if f(p) ≠ p′
9      else
10         raise Mismatch if τ(p′) ⊀ τ(p)
11         raise Mismatch if p′ ∈ ran(f)
12         f ← f ∪ {p ↦ p′}
13         forall p -a-> q ∈ E do
14             raise Mismatch if p′ -a-> q′ ∉ E′
15             dfs_match(E, q, E′, q′)
16         end
17     end
18 end
```

construct the match morphism during the DFS. If the construction fails, we can conclude that the match morphism does not exist.

Algorithm 3.1 gives the precise construction procedure. It takes the preorder and type-assignment on the nodes and two layout graphs as the input, and output a morphism mapping between the nodes of the two graphs, if the match can be established, otherwise an exception is raised. The procedure initializes the morphism to an empty map, and starts traversing both layout graphs synchronously from their corresponding roots. The dfs_match function performs the DFS. It checks the 1-1 mapping and the preorder relation between the two roots, and accumulates the mapping to the morphism. Then the function checks, w.r.t. the edge labels, the correspondence of the adjacent nodes, and recurs on each pair of them.

3.3 Pattern Graphs

A pattern graph $P = \langle G, T \rangle$ is a layout graph P with a map T of type-assignment, which associates each node with a type. A type can be a usual class, a subclass, a set of values and particularly a singleton set of one value. The preorder $\prec$ on the node types of such pattern graphs is exactly the subtype relation between two types. In principle, a type t' is a subtype of another type t if all objects or values of t' are elements of t, i.e.,

Table 3.1: Subtyping Rules

$$\text{(Reflexive)}\ \frac{c \in \mathscr{C}}{c \prec c} \qquad \text{(Subclass)}\ \frac{c, d \in \mathscr{C} \quad \Gamma \vdash d \rightarrowtriangle c}{\Gamma \vdash d \prec c} \qquad \text{(Subset)}\ \frac{W \subseteq V \subseteq \mathscr{V}}{W \prec V}$$

$$\text{(Value-Type)}\ \frac{V \subseteq \mathscr{V} \quad t \in \mathscr{C} \quad \Gamma \vdash \forall v \in V \bullet v : t}{\Gamma \vdash V \prec t},$$

elements$(t') \subseteq$ elements(t).

Definition 6 (Subtype) A type is either a class $c \in \mathscr{C}$ or a set of values $V \subset \mathscr{V}$. The subtype relation $\prec$ is inferred from the typing environment Γ by the subtyping rules given in Table 3.1, where $\rightarrowtriangle$ is the subclass relation.

Theorem 4 The subtype relation $\prec$ in Definition 6 is a preorder.

Proof. All we need is that the subclass relation $\rightarrowtriangle$ being unidirectional and transistive, and so it is. □

Definition 7 (Pattern Match) Given two pattern graphs $P = \langle G, T \rangle$, $P' = \langle G', T' \rangle$ and typing environment Γ, P' matches P w.r.t. preorder $\prec$ on node types, denoted as $P' \sqsubset|_{\prec} P$, if $G' \sqsubset|_{\prec} G$, where $\tau(u') \prec \tau(u) \triangleq \Gamma \vdash T'(u') \prec T(u)$.

An object corresponds to a node in a pattern graph, with the root representing the main object, and the edges leading to component objects. In most cases, an object must terminate at some value nodes, which are primitives in a programming language, such as integers and booleans. It is common to use the value itself as the node. This formulation complicates the graph model by having concrete and multiple types of nodes. To keep the nodes pure and abstract, we associate a node p with its value value(p) through a run-time function. We then store this value as a singleton set of one value when constructing the type map of a pattern graph. The real type of the node can be obtained from the value, since any terminus does not start a structure, and must have a known type in the type system.

The state Σ of a running program can be represented by a layout graph H, together with the above mentioned value function and the run-time type information, where H consists of a linked list of scope nodes as a stack, with outgoing edges as local variables to the roots of the objects [3]. Therefore, a pattern graph can be extracted from a state by re-rooting H at the root p of the object. The map of type assignment is constructed from type(p), which returns the actual type of the object.

Definition 8 (Pattern Extraction) A pattern graph $P = \langle G, T \rangle$ is extracted from a state $\Sigma = \langle H, \text{value}, \text{type} \rangle$ at node p, denoted as $P = \Sigma \odot p$, where $G = H \odot p$, and for all

$u \in \text{nodes}(G)$,

$$T(u) = \begin{cases} \text{type}(u) & \text{if } u \notin \text{dom}(\text{value}), \\ \{\text{value}(u)\} & \text{if } u \in \text{dom}(\text{value}). \end{cases}$$

3.4 Match Conjunction and Disjunction

When we use pattern matching, it is natural to think about matching one object against multiple patterns, and take the conjunction or disjunction of the individual matches. When taking the conjunction of multiple patterns, we want to know if their overlapping parts are consistent. This observation can tell us if it is possible to have some object matching all these patterns simultaneously. On the other hand, when taking the disjunction of multiple patterns, the overlapping parts give us the common component of the individual matches. This component carries the properties that we can ensure no matter which branch succeeds.

To check whether multiple pattern graphs are consistent, we try to construct a union graph that matches these layouts. The construction is divided into two parts. First, we make sure that an isomorphism is possible from each layout to the union. This tells whether the patterns are structurally consistent. Second, we make sure that there exists at least one type that is a subtype of all the types associated with the corresponding nodes of the patterns, better there exists an infimum. This tells whether the patterns are consistent in the type system.

Aliasing is an inherit feature of object references, if a node is a common target of two edges, it can be mapped to two nodes of different layouts separately. However, there is no way for this node to be mapped to the two nodes of the union graph simultaneously. We need to address the aliasing issue when we use the union graph to deal with match conjunction.

Definition 9 (De-aliasing) Let $G = \langle N, E, p\rangle$ and $G' = \langle N', E', p'\rangle$ be two layout graphs. G is a de-aliased graph of G', denoted as $G \rightrightarrows_f G'$, if there exists a total function $f : N \to N'$, such that

1. root is preserved, $f(p) = p'$,
2. edges are preserved, $u \xrightarrow{a} v \in E \implies f(u) \xrightarrow{a} f(v) \in E'$ and $u' \xrightarrow{a} v' \in E' \implies \exists u \xrightarrow{a} v \in E \bullet (u', v') = (f(u), f(v))$.

For a layout graph G'', if $\exists G \rightrightarrows G' \bullet G'' \gtrsim G$, we say G'' covers some de-aliased graph of G', denoted as $G'' \gtrsim\!\!\rightrightarrows G'$. If $\exists G \rightrightarrows G' \bullet G'' \lesssim G$, we say G'' is covered by some de-aliased graph of G', denoted as $G'' \lesssim\!\!\rightrightarrows G'$.

Definition 10 (Union) $G_\diamond$ is a union of multiple layout graphs $\overline{G}$, denoted as $G_\diamond \cong_f \bigcup \overline{G}$, if $\overline{G \lesssim_f G_\diamond}$, and for all G' such that $\overline{G \lesssim G'}$, we have $G_\diamond \lesssim\!\!\rightrightarrows G'$.

Algorithm 3.2: Construction of Union

Input: layout graphs: $\overline{G}$.
Output: morphisms to union: $\overline{f}$.
Exception: *No-Union*.

begin
  $\overline{\langle N, E, p\rangle = G}$
  $\overline{f \leftarrow \emptyset}$
  dfs_union$(\overline{E}, \overline{p})$
end

function dfs_union$(\overline{E_i}, \overline{p_i})$ **begin**
  $\overline{k} = \{k \in \overline{i} \mid p_k \in \mathrm{dom}(f_k)\}$
  $\overline{c} = \overline{i} \setminus \overline{k}$
  $\overline{u = f_k(p_k)}$
  raise *No-Union* **if** $|\{\overline{u}\}| > 1$
  if $\overline{c} \neq \emptyset$ **then**
    if $\overline{u} \neq \emptyset$ **then**
      raise *No-Union* **if** $u_1 \in \bigcup \overline{\mathrm{ran}(f_c)}$
      $p_\diamond \leftarrow u_1$
    else
      $p_\diamond \leftarrow$ new node
    end
    $\overline{f_c \leftarrow f_c \cup \{p_c \mapsto p_\diamond\}}$
    forall $a, \overline{j} = \{j \in \overline{i} \mid p_j \xrightarrow{a} q_j \in E_j\}$ **do**
      dfs_union$(\overline{E_j}, \overline{q_j})$
    end
  end
end

The union is the minimum layout graph that covers all the given layouts, and cannot be further de-aliased.

Theorem 5 Given multiple layout graphs $\overline{G}$, if there exists a layout graph G' that covers all of them, there exists the union of $\overline{G}$.

Proof. Since we have $\overline{G \lesssim_f G'}$, we can construct the union $G_\diamond$ from $\overline{f}$ and $\overline{G}$. The nodes of $G_\diamond$ are mapped from $\overline{\mathrm{dom}(f)}$, and combined only when necessary. If a node u_i in $\mathrm{dom}(f_i)$ has the set of incoming paths intersecting with that of a node u_j in $\mathrm{dom}(f_j)$, both u_i and u_j should be mapped to one node [4]. Otherwise, they should be mapped to different nodes. The existence of $\overline{f}$ guarantees the set of incoming paths of a node in one layout will not lead to more than one node in another layout. Such construction splits the target nodes whenever possible, ensuring that the result cannot be further de-aliased. □

As illustrated in Algorithm 3.2, we try to construct the morphisms $\overline{f}$ from each graph to the union by traversing the graphs simultaneously in DFS, while checking for conflicts.

In each iteration of the DFS, we collect the nodes $\overline{p_i}$ that have a common incoming path only in those graphs $\overline{G_i}$, where $\overline{G_i}$ is a subset of $\overline{G}$. These nodes $\overline{p_i}$ should be mapped to the same node in the union. We divide $\overline{p_i}$ into two sets, one set $\overline{p_k}$ are previously visited from other paths, the other set $\overline{p_c}$ are not yet visited. There are two conflicts to check for. First, the visited nodes must all be mapped to one node u. Second, this u must not be an existing target from those graphs $\overline{G_c}$ where the unvisited nodes reside, which means there are two paths leading to one node in $\overline{G_k}$, however leading to two nodes in $\overline{G_c}$. If no conflicts are detected, we assign u, or a new node if u does not exist, as the mapping target of those unvisited nodes $\overline{p_c}$.

Definition 11 (Intersection) $G_\circ$ is an intersection of multiple layout graphs $\overline{G}$, denoted as $G_\circ \cong_{f\circ g} \bigcap \overline{G}$, if $\overline{G \gtrsim_f \rightrightarrows_g G_\circ}$, and for all G' such that $\overline{G \gtrsim \rightrightarrows G'}$, we have $G_\circ \gtrsim \rightrightarrows G'$.

The intersection is the maximum layout graph that all the given layouts cover some de-aliased version of it.

Theorem 6 Given multiple layout graphs $\overline{G}$, there always exists the intersection of $\overline{G}$.

Proof. We can construct the intersection as follows. First, the intersection begins with collecting those incoming paths common in all $\overline{G}$. Then, if any two paths reach the same node in any one of $\overline{G}$, we combine them into one set of incoming paths, otherwise, we keep them as two separate sets. Finally, each set of incoming paths is mapped to a node of the intersection. Similar to the union, such construction splits the target nodes whenever possible, ensuring that the result cannot be further de-aliased. □

As illustrated in Algorithm 3.3, the construction of intersection is simpler. We again traverse the graphs $\overline{G}$ simultaneously in DFS. In each iteration, we collect only those node $\overline{p}$ with a common incoming path in all the given layouts. If some of the nodes $\overline{p_k}$ are previously visited, they may have been mapped to multiple nodes $\overline{u}$. We take one of the targets u_1 to unify the mapping target of all these nodes. If all $\overline{p}$ are unvisited, we map all of them to a new node.

The union and intersection give us the common structures of multiple layout graphs, however without respect to the preorder between node types. If there is an object matching all the given layouts $\overline{G}$, it must contain a subgraph G' that can be de-aliased to the union of $\overline{G}$. Each node u' of G' corresponds to one node $u_\diamond$ of the union in each component match of the conjunction. This $u_\diamond$ must therefore be of a subtype of all the nodes mapped to it from $\overline{G}$. We can assume a better type of $u_\diamond$ and thus u', if there exists an infimum type of all the nodes mapped to $u_\diamond$.

Definition 12 (Match Conjunction) G' matches the conjunction of layout graphs $\overline{G}$, w.r.t. preorder $\prec$, denoted as $G' \sqsubset|_\prec \curlywedge\overline{G}$, if $\overline{G' \sqsubset|_\prec G}$.

Algorithm 3.4 constructs the type assignment of the union based on the given patterns. We first obtain the union morphisms from the layouts by calling Algorithm 3.2.

Algorithm 3.3: Construction of Intersection

Input: layout graphs: $\overline{G}$.
Output: morphisms to intersection: $\overline{f}$.

```
1  begin
2      ⟨N, E, p⟩ = G
3      f ← ∅
4      dfs_inter(E, p)
5  end
6  function dfs_inter(E, p) begin
7      k = {k | p_k ∈ dom(f_k)}
8      u = f_k(p_k)
9      if u ≠ ∅ then
10         p_◦ ← u_1
11     else
12         p_◦ ← new node
13     end
14     f ← f ∪ {p ↦ p_◦}
15     if |u| ≠ |p| then
16         forall a • p -a-> q ∈ E do
17             dfs_inter(E, q)
18         end
19     end
20 end
```

Then, for each node $u_\diamond$ in the union, we collect all the layouts having nodes mapped to $u_\diamond$. These layouts correspond to morphisms $\overline{f_k}$ and type assignments $\overline{T_k}$. Thus, we are able to look up the types $\overline{t_k}$ associated with the nodes mapped to $u_\diamond$, and check for the legitimacy of subtyping $\overline{t_k}$. If the typing environment does not permit such subtyping, we must conclude that the conjunction of these patterns are not possible. If the subtyping is possible, we try to find the infimum $t_\diamond$ of $\overline{t_k}$ and associate it with $u_\diamond$. The type of $u_\diamond$ is *No-Type* if there exists no infimum.

On the contrary, for match disjunction, an object is to match only one of the multiple patterns. This is always possible because neither structural nor subtyping conflicts can occur for only one graph. Any object that matches one of the given layouts $\overline{G}$ must contain a subgraph G' which is a de-aliased graph of the intersection of $\overline{G}$. Each node u' of G' is mapped to the corresponding node $u_\circ$ of the intersection possibly via any component match of the disjunction. This $u_\circ$ must therefore be of a supertype of all the nodes mapped to it, to cope with all the possible individual matches. Even if such a supertype exists, it is possibly not unique. We can only associate a type with $u_\circ$ when there is a supremum type of all the nodes mapped to $u_\circ$.

Definition 13 (Match Disjunction) G' matches the disjunction of layout graphs $\overline{G}$,

Algorithm 3.4: Construction of Match Conjunction

Input: typing environment: Γ, pattern graphs: $\overline{P}$.
Output: morphisms to union: $\overline{f}$ and type assignment: $T_\diamond$.
Exception: *No-Union*, *No-Conj*.

begin
  $\overline{\langle G, T\rangle = P}$
  Algorithm 3.2: $\overline{G} \rightarrow \overline{f}$
  $T_\diamond \leftarrow \emptyset$
  $N_\diamond = \bigcup \overline{\mathrm{ran}(f)}$
  forall $u_\diamond \in N_\diamond$ **do**
    $\overline{k} = \{k \mid u_\diamond \in \mathrm{ran}(f_k)\}$
    $\overline{t_k = T_k(f_k^{-1}(u_\diamond))}$
    raise *No-Conj* **if** $\Gamma \vdash \nexists t \bullet \overline{t \prec t_k}$
    $\Gamma \vdash t_\diamond = \inf(\overline{t_k})$
    $T_\diamond \leftarrow T_\diamond \cup \{u_\diamond \mapsto t_\diamond\}$
  end
end

w.r.t. preorder $\prec$, denoted as $G' \sqsubset|_\prec \curlyvee \overline{G}$, if there exists $G'' \in \overline{G}$ such that $G' \sqsubset|_\prec G''$.

Algorithm 3.5 constructs the type assignment of the intersection based on the given patterns. We first obtain the intersection morphisms from the layouts by calling Algorithm 3.3. Then, for each node $u_\circ$ in the intersection, we find the nodes mapped to $u_\circ$ in all the patterns. Note that a morphism to the intersection is not injective, there can be more than one nodes mapped to $u_\circ$. Next, we look up the types $\overline{t}$ associated with these nodes. Finally, we try to find the supremum $t_\circ$ of $\overline{t}$ and associate it with $u_\circ$. The type of $u_\circ$ is *No-Type* if there exists no supremum.

Algorithm 3.5: Construction of Match Disjunction

Input: typing environment: Γ, pattern graphs: $\overline{P}$.
Output: morphisms to intersection: $\overline{f}$ and type assignment: $T_\circ$.

begin
  $\overline{\langle G, T\rangle} = P$
  Algorithm 3.3: $\overline{G} \longrightarrow \overline{f}$
  $T_\circ \longleftarrow \emptyset$
  $N_\circ = \bigcup \overline{\mathrm{ran}(f)}$
  forall $u_\circ \in N_\circ$ **do**
    $\overline{t} = \bigcup \overline{\{T(u) \mid f(u) = u_\circ\}}$
    $\Gamma \vdash t_\circ = \sup(\overline{t})$
    $T_\circ \longleftarrow T_\circ \cup \{u_\circ \mapsto t_\circ\}$
  end
end

Chapter 4.

PATTERN LANGUAGE

Contents

4.1 Syntax 23
4.2 Pattern Graph Construction 24
4.3 Type-Checking 25
4.4 Operational Semantics 26
4.5 Type Safety 27

With the detail of the graph model for patterns and layouts fully discussed. We give a language extension for pattern matching to general OO languages. We then present the type-checking rules and operational semantics, and prove the type safety to reason about the soundness of our model.

4.1 Syntax

The abstract syntax to define patterns and matches is given in Table 4.1. The syntax is much like conventional ones found in other OO and functional languages, with the addition of pattern references. The pattern references play two roles, to be referred to from inside the pattern to produce aliases and cycles, and to be referred to from the match-selected statement as object bindings — variables bound to components of matching objects.

A pattern P can have an optional label, which is a pattern reference r, followed by a pattern specification in one of the three forms — a class pattern, a pattern reference or a set V of values. A class pattern K is a class type T followed by zero or more attribute specifications, each of which is an attribute name a equal to an inner pattern P. For example, the pattern p_4 shown in Figure 1.3 can be expressed as

$$T_1(b = T_2, c = y : T_3, d = y).$$

The more complex pattern shown in Figure 1.2 is written as

$$T(l = w : T(l = x : T, r = y : T(l = w, r = z)), r = z : T).$$

A match statement evaluates an expression e and matches the result against many cases in turn. Each case consists of a pattern junction J, which can be a single pattern, a

Table 4.1: Abstract Syntax of Pattern Matching

P	::=	$[r :] \ (K \mid r \mid V)$	pattern
K	::=	$T \ \overline{a = P}$	class pattern
r			pattern reference
V			value set
T			class type
a			attribute
J	::=	$P \mid \curlywedge \overline{P} \mid \curlyvee \overline{P}$	pattern junction
M	::=	$e \ltimes \overline{J \rightarrow S}$	match statement
S			statement

pattern conjunction $\curlywedge \overline{P}$ or a pattern disjunction $\curlyvee \overline{P}$, and an associated statement S. The first successfully matched case selects the associated statement to execute, with the new object bindings introduced by the references declared in the matched pattern junction.

4.2 Pattern Graph Construction

To carry out the operations of pattern matching and check for its type safety based on the graph model described in Chapter 3, we must first construct the pattern graph for each pattern specified in the abstract syntax. The construction is straightforward following the structure of the specification, with three components to output — a layout graph G, a map T from nodes to types and a map R from pattern references to nodes. Algorithm 4.1 describes the procedure in detail, where, for clarity, we enclose syntax elements in guillemets « » to distinguish them from other variables.

In the algorithm, function parse recursively decomposes a pattern specification «P» into a graph with root p by updating the nodes N, the edges E, the type map T and the reference map R. The parse function performs case analysis on the pattern specification. First, when it is a reference definition «$r : Q$», we recursively obtain the root p of «Q», and associate it with the reference «r» in R. It is possible for a reference to be used before definition, we generate a placeholder node r for the forward use, and replace it with the real node at this definition stage. A placeholder is not put into the node set N, therefore we can check if r is indeed a placeholder, or a node that has defined the reference already. For a placeholder r, we need to substitute the real node p for it in the edges E, for all the source and target nodes equal to r, and also in the reference map R, for all the image nodes equal to r. Note that we allow a reference to label another reference. Second, when it is a class pattern «$T \ \overline{a = Q}$», we create a new node p and associate it with type «T». We add p to the node set N and link the outgoing edges to the roots recursively obtained from $\overline{\text{«}Q\text{»}}$, with labels equal to the attribute names $\overline{\text{«}a\text{»}}$. Third, when it is a reference occurrence «r», we try to look up R for its corresponding

node, or create a new placeholder without placing it into the node set N. Last, when it is a value set «V», we simply create a new node and associate it with «V» in the type map T.

The graph construction merely creates nodes and links between the nodes. This process is always successful. However, with the addition of references, there are two exceptional cases — a reference is defined more than once (*Redef-Ref*), and a reference occurs but not defined (*Undef-Ref*). The former is detected at the time of parsing a label prefix, while the latter is detected when the parsing is complete, by checking whether all the placeholders have been resolved.

4.3 Type-Checking

The pattern matching extension is type-checked in two parts, the pattern and the match. A pattern can be a class pattern, a reference, a pattern conjunction or a pattern disjunction. A class pattern is converted to a pattern graph and checked by recursively reducing to each node and the outgoing edges. A node in a pattern graph is well-typed when all its outgoing edges are labeled with the attributes of the associated class, and also the targets are all well-typed. To avoid infinite loops in the DFS, we add the visited nodes to the typing environment before going into recursion. A reference is checked by resolving it to a node with the reference map. A class pattern is well-typed only when the corresponding pattern graph and all the references declared in the pattern are well-typed.

The type-checking rules are given in Table 4.2. Each typing rule involves a typing environment Γ and optionally some additional structures, separated by a semicolon, as the context. The result of a rule is the well-typedness and type-assignment of the checked element, possibly with some extra output, also separated by a semicolon, during the checking. This output can be necessary in the subsequent procedures.

For a pattern conjunction, we check the individual patterns first, and try to construct the match conjunction. Then, we promote the type of each referenced node to the infimum type associated with the union. If such infimum type does not exist, we fall back to the original type of the node. For a pattern disjunction, we try to construct the match disjunction. Only a referenced node that occurs commonly in all the component patterns has a type, which must be the supremum type associated with the intersection. If such supremum type does not exist, the referenced node has *No-Type*, and subsequent uses of the reference will be ill-typed. This includes the case where a referenced node does not occur in all the component patterns.

A match is a test of an expression against multiple cases. Therefore, to type-check a match, we need to check the individual cases, and whether the expression is type-compatible with all the cases. Each case consists of a pattern and a statement. The checking of the pattern produces the type of the pattern and a series of type-assigned

references, as the additional variables available to the statement. We check the type of the pattern against the expression to match, and check the statement under the context with those additional variables declared.

4.4 Operational Semantics

The operation of a pattern match statement is to evaluate the expression, then to match the result against the list of cases in turn, and finally to select the statement in the matched case to execute. In the graph model, the expression «e» to match is evaluated to a node in the state graph, and then the pattern graph P rooted at this node is extracted from the state graph. The pattern graph is matched in turn against all the cases until a matched case is found, otherwise there is no statement to execute in this match.

The operational semantics is given by the state transition rules listed in Table 4.3. Before we can match the pattern graph of the expression, we need to obtained the pattern graphs of the patterns by taking the output of the type-checking procedures. Beside to match by constructing the match morphisms, we also need to obtain the references declared in the patterns, as well as their corresponding nodes. This task is done in the match function, divided into three cases, a single pattern, a conjunction, or a disjunction. For a single pattern, a reference r is mapped to a node of the pattern graph P_1 by the reference map R_1, then we can use the match morphism f from the pattern graph to the expression graph P to obtain the corresponding node in the state graph. For a conjunction, since the match of the expression graph must be done against all the component pattern graphs, we have a series of match morphisms from different pattern graphs to the same expression graph, their domains do not intersect with each other. We can simply union these morphisms into one to obtain the nodes of the references.

A disjunction is more complicated, because there requires only one match of the component pattern graphs, and a node in the intersection is possibly de-aliased to two or more nodes in a component pattern. We have restricted the references in a disjunction only to those nodes in the intersection, by assigning *No-Type* to the other nodes. Therefore, we are left with a one-to-many mapping from the intersection to the matched pattern, and thus to the expression graph. We use the non-deterministic choice $\sqcap$ to bind a reference to one of the possible corresponding nodes in the state graph. Note that, in function match, f_1' is the union of the morphisms from the patterns to their intersection. Once we have the bindings of the references to the nodes in the state graph, we can declare the additional variables as a normal variable declaration, **var** ... **end**, and execute the statement of the matched case in this scope.

4.5 Type Safety

The type safety of pattern matching means a well-typed pattern match statement cannot go wrong at runtime. Since the only new operation introduced by pattern matching is the new bindings of pattern references to runtime objects, the statement of the matched case is type-safe when the runtime objects truly have the types associated with the nodes of the matched pattern.

Theorem 7 For the same pattern junction $\langle\!\langle J \rangle\!\rangle$, let $(\overline{r}, \overline{q})$ be the result of function match in Table 4.3, $\overline{r : s}$ the type assignment obtained by (Case) in Table 4.2. We have $\overline{\mathrm{type}(q) \prec s}$.

Proof. We study the type t of the node q associated to a reference r in function match by case analysis. In the case of (Match-One), t comes from the type assignment of pattern graph P. By the definition of $\sqsubset_f$, $t \prec s$, where s is the type of node $R_1(r)$ in pattern graph P_1, such that $q = f(R_1(r))$. In fact, in (Match-Some), P_1 and R_1 are obtained from the type-checking rules. According to (Pattern), (Graph) and (Node), s is exactly the type assigned to r in the type-checking.

Similarly, in the case of (Match-Conj), P matches all $\overline{P_1}$. The type t of q is a subtype of all the types $\overline{t_k}$ of the corresponding nodes in each of $\overline{P_1}$. By Algorithm 3.4, if there exists $\inf(\overline{t_k})$, it is assigned to r in (Conj), for $\inf(\overline{t_k})$ is a subtype of each $\overline{t_k}$, and t must be a subtype of the infimum by definition. It there exists no infimum, *No-Type* is assigned to the node in the union, thus the $\overline{\min(\textit{No-Type}, t_k)}$ in (Conj) assigns r with one of $\overline{t_k}$, of which t is a subtype.

In the case of (Match-Disj), P matches only one graph P' of $\overline{P_1}$. The node q bound to r is non-deterministically chosen from nodes $\overline{f(u)}$ in P, matching those nodes $\overline{u}$ in P' that are finally mapped to the same node $u_\circ$ in the intersection of $\overline{P_1}$. The type t of q is a subtype of the corresponding node in P' mapped to $u_\circ$. By Algorithm 3.5 and (Disj), r is assigned with the supremum type of all the node in $\overline{P_1}$ mapped to $u_\circ$, or *No-Type* if the supremum does not exist. Obviously, in either case, t is a subtype of the type assigned to r. □

Algorithm 4.1: Construction of Pattern Graph

Input: pattern: $«P»$.
Output: layout graph: G, type map: T and reference map: R.
Exception: *Undef-Ref*, *Redef-Ref*.

begin
  $N, E, T, R \leftarrow \emptyset$
  $p \leftarrow$ parse$(«P»)$
  raise *Undef-Ref* **if** $\mathrm{ran}(R) \not\subseteq N$
  $G \leftarrow \langle N, E, p \rangle$
end

function parse$(«P»)$ $\rightarrow p$ **begin**
  if $«P \Rightarrow r : Q»$ **then**
    $p \leftarrow$ parse$(«Q»)$
    if $«r» \in \mathrm{dom}(R)$ **then**
      $r \leftarrow R(«r»)$
      raise *Redef-Ref* **if** $r \in N$
      $E \leftarrow E[p/r], R \leftarrow R[p/r]$
    end
    $R \leftarrow R \uplus \{«r» \mapsto p\}$
  else if $«P \Rightarrow T\ \overline{a = Q}»$ **then**
    $p \leftarrow$ new node
    $N \leftarrow N \cup \{p\}$
    $T \leftarrow T \cup \{p \mapsto «T»\}$
    $\overline{q \leftarrow \text{parse}(«Q»)}$
    $E \leftarrow E \cup \{p \xrightarrow{«a»} q\}$
  else if $«P \Rightarrow r»$ **then**
    if $«r» \in \mathrm{dom}(R)$ **then**
      $p \leftarrow R(«r»)$
    else
      $p \leftarrow$ new node
      $R \leftarrow R \cup \{«r» \mapsto p\}$
    end
  else if $«P \Rightarrow V»$ **then**
    $p \leftarrow$ new node
    $N \leftarrow N \cup \{p\}$
    $T \leftarrow T \cup \{p \mapsto «V»\}$
  end
end

Table 4.2: Type-Checking Rules

$$\text{(Node)}\ \frac{\Gamma \vdash t = T(p) \quad \overline{p \xrightarrow{a} q \in E} \quad p : t, \Gamma;\ E, T \vdash \overline{q : s} \quad \Gamma \vdash \overline{t.a : s' \wedge s \prec s'}}{\Gamma;\ E, T \vdash p : t}$$

$$\text{(Graph)}\ \frac{P = \langle N, E, p, T\rangle \quad \Gamma;\ E, T \vdash p : t}{\Gamma \vdash P : t}$$

$$\text{(Ref)}\ \frac{R(r) = p \quad P = \langle N, E, p, T\rangle \quad \Gamma; E, T \vdash p : t}{\Gamma;\ P, R \vdash r : t}$$

$$\text{(Pattern)}\ \frac{\text{Algorithm 4.1} : \text{«}P\text{»} \longrightarrow G, T, R \quad P = \langle G, T\rangle \quad \Gamma \vdash P : t \quad \overline{r} = \mathrm{dom}(R) \quad \Gamma;\ P, R \vdash \overline{r : s}}{\Gamma \vdash \text{«}P\text{»} : t, \overline{r : s};\ P, R}$$

$$\text{(Conj)}\ \frac{\Gamma \vdash \overline{\text{«}P\text{»} : t, \overline{r : s};\ P, R} \quad \text{Algorithm 3.4} : \Gamma, \overline{P} \longrightarrow \overline{f}, T_\diamond \quad \overline{s' = \min(s, T_\diamond(f(R(r))))} \quad \hat{t} = T_\diamond(f_1(R_1(\mathrm{root}(P_1)))) \quad \overline{t' = \min(t, \hat{t})}}{\Gamma \vdash \text{«}\curlywedge\overline{P}\text{»} : \hat{t}, \overline{\text{«}P\text{»} : t'}, \overline{r : s'};\ \overline{P}, \overline{f}, \overline{R}}$$

$$\text{(Disj)}\ \frac{\Gamma \vdash \overline{\text{«}P\text{»} : t, \overline{r : s};\ P, R} \quad \text{Algorithm 3.5} : \Gamma, \overline{P} \longrightarrow \overline{f}, T_\circ \quad \overline{s' = T_\circ(f(R(r)))} \quad \hat{t} = T_\circ(f_1(R_1(\mathrm{root}(P_1))))}{\Gamma \vdash \text{«}\curlyvee\overline{P}\text{»} : \hat{t}, \overline{\text{«}P\text{»} : t}, \overline{r : s'};\ \overline{P}, \overline{f}, \overline{R}}$$

$$\text{(Case)}\ \frac{\Gamma \vdash \text{«}J\text{»} : t, \overline{r : s};\ \overline{P}, \overline{f}, \overline{R} \quad \overline{r : s}, \Gamma \vdash \text{«}S\text{»}}{\Gamma \vdash \text{«}J \longrightarrow S\text{»} : t}$$

$$\text{(Match)}\ \frac{\Gamma \vdash \overline{\text{«}J \longrightarrow S\text{»} : t} \quad \Gamma \vdash e : t' \quad \Gamma \vdash \overline{t' \prec t \vee t \prec t'}}{\Gamma \vdash \text{«}e \ltimes \overline{J \longrightarrow S}\text{»}}$$

Table 4.3: State Transition Rules

$$\text{(Match)}\ \frac{\mathrm{eval}(\langle\!\langle e \rangle\!\rangle) = p \qquad P = \Sigma \odot p}{\langle\!\langle e \ltimes \overline{J \longrightarrow S} \rangle\!\rangle, \Sigma\rangle \longrightarrow \langle P \ltimes \langle\!\langle \overline{J \longrightarrow S} \rangle\!\rangle, \Sigma\rangle} \qquad \text{(Match-None)}\ \langle P \ltimes \langle\!\langle \emptyset \rangle\!\rangle, \Sigma\rangle \longrightarrow \Sigma$$

$$\text{(Match-Some)}\ \frac{\langle\!\langle \overline{J \longrightarrow S} \rangle\!\rangle = \langle\!\langle J_1 \longrightarrow S_1, \overline{J' \longrightarrow S'} \rangle\!\rangle \qquad \Gamma \vdash \langle\!\langle J_1 \rangle\!\rangle;\ \overline{P_1}, \overline{f_1}, \overline{R_1}}{\mathrm{match}(P, \langle\!\langle J_1 \rangle\!\rangle, \overline{P_1}, \overline{f_1}, \overline{R_1}) = (\overline{r}, \overline{q})}$$

$$\frac{}{\langle P \ltimes \langle\!\langle \overline{J \longrightarrow S} \rangle\!\rangle, \Sigma\rangle \longrightarrow \langle\!\langle \mathbf{var}(\overline{r = q}); S; \mathbf{end}(\overline{r}) \rangle\!\rangle, \Sigma\rangle}$$

$$\frac{\mathrm{match}(P, \langle\!\langle J_1 \rangle\!\rangle, \overline{P_1}, \overline{f_1}, \overline{R_1}) = \emptyset}{\langle P \ltimes \langle\!\langle \overline{J \longrightarrow S} \rangle\!\rangle, \Sigma\rangle \longrightarrow \langle P \ltimes \langle\!\langle \overline{J' \longrightarrow S'} \rangle\!\rangle, \Sigma\rangle}$$

$$\mathrm{match}(P, \langle\!\langle J_1 \rangle\!\rangle, \overline{P_1}, \overline{f_1}, \overline{R_1}) \triangleq$$

$$\text{(Match-One)}\ \frac{\langle\!\langle J_1 \rangle\!\rangle = \langle\!\langle P \rangle\!\rangle \qquad \Gamma \vdash P \sqsubset_f P_1}{\overline{r} = \mathrm{dom}(R_1) \qquad \overline{q = f(R_1(r))}}$$

$$\text{(Match-Conj)}\ \frac{\langle\!\langle J_1 \rangle\!\rangle = \langle\!\langle \curlywedge \overline{P} \rangle\!\rangle \qquad \Gamma \vdash \overline{P \sqsubset_f P_1} \qquad R' = \bigcup \overline{R_1} \qquad f' = \bigcup \overline{f}}{\overline{r} = \mathrm{dom}(R') \qquad \overline{q = f'(R'(r))}}$$

$$\text{(Match-Disj)}\ \frac{\langle\!\langle J_1 \rangle\!\rangle = \langle\!\langle \curlyvee \overline{P} \rangle\!\rangle \qquad \Gamma \vdash \exists P' \in \overline{P_1} \bullet P \sqsubset_f P' \qquad R' = \bigcup \overline{R_1} \qquad f_1' = \bigcup \overline{f_1} \qquad \overline{u_\circ = f_1'(R'(r))}}{\overline{r} = \mathrm{dom}(R') \qquad \overline{q = \bigsqcap \{f(u) \mid f_1'(u) = u_\circ\}}}$$

$$\text{(Mismatch)}\ \frac{\text{otherwise}}{\emptyset}$$

Chapter 5.

DISCUSSION

We formalize pattern matching in the notions of object graphs. Although our intention is to specify patterns according to the underlying stored structures, the nodes, edges, labels and preorder are in fact abstract, thus can represent a wider range of relations between objects. For example, we can replace concrete attributes with getter methods. The patterns to match will not change in this case, and the pattern graph of the object to examine can be obtained by first calling the getters. Even if side effects exist, the semantics of evaluating the getters can be made unambiguous, possibly imitating lazy evaluation. Edge labels can be parameter positions of a decomposition function, patterns with such positional labels can be matched by the tuple result of the decomposition.

The nodes in our pattern graphs are pure identities. Value objects have to be represented as pure nodes associated with singleton sets. Since we treat value sets as types, matching values is classified as matching types. Because not only an identical type matches, a subtype also matches, this gives the benefit to match a value against a range of values, for a subset can be seen as a subtype. For example, pattern $T(e = [5, \ldots, 9])$ is matched by a tree node whose element is between 5 and 9. This arrangement supersedes the traditional matching of value cases.

The mechanism to specify and detect aliasing is a distinct feature of our system. However, it also brings certain restriction. Sometimes we want to ignore the aliasing by intention, especially for immutable objects whose aliases are safe. For example, we may write a pattern $T(c = S, d = S)$ to test if an object has two components both of type S. While an object with layout p_3 in Figure 1.3 matches the pattern, but an object with layout p_4 does not. This causes some inconvenience. In the case to ignore aliasing, we can try match conjunction $T(c = S) \wedge T(d = S)$ to split the aliased node into two component tests. However, we cannot use match conjunction to ignore cycles, for example, the layout in Figure 1.2 does not match pattern $T(l = T(r = T(l = T)))$, because there's no way to split a node that is both a source and a target into two tests. A better solution is to construct an intermediate object graph to split (de-alias) the nodes that are to ignore aliases and cycles beforehand, and match the intermediate graph against the pattern. This can be optimized into a slightly modified match algorithm, based on Algorithm 3.1, to split the nodes on the fly in construction of the match morphism.

The subtyping in our pattern matching only needs to be a preorder in the match of a single pattern. In match conjunction, the node of an object to match must has a subtype of multiple types. In the classical single inheritance hierarchy, unless these types form an inheritance chain, the match cannot succeed. Modern OO languages all have interfaces

that support multiple inheritance, therefore, a subtype of multiple unrelated types is possible. However, there may exist no infimum. For a match conjunction $T(a = x : I) \wedge T(a = J)$, we may have A and B both extending I and J, but the common $\inf(I, J)$ does not exist. In this case, we must fall back the type of x to I even if we know that x must also have type J when a match succeeds. In languages supporting intersection types [30], $\inf(I, J) = I \cap J$ can be generated and assigned to x for better typing.

On the other hand, to find the type of a variable in match disjunction is even more complicated. Consider the following disjunction,

$$p : T(a = I, b = J, c = x : X) \vee q : T(a = y : K, b = y).$$

Not like in a dynamic language such as Python, where using undefined variables is allowed to raise run-time exceptions, for a statically typed language, we must guarantee that a program which passes type-checking must not have such exceptions. First, we must reject the use of any variable like x which is not in the common part of the disjunction. This is done by assigning *No-Type* to x. Second, we must collect all the types of a node that possibly occurs in any part of the disjunction with the same incoming path. For y, the set of incoming paths is $\{[a], [b]\}$, therefore, the type of y must be a supertype of all the types reachable from $\{[a], [b]\}$. The best choice of this supertype is the supremum, $\sup(I, J, K)$, if it exists. Otherwise, we choose *No-Type* to reject y, in fact, although nondeterministic, any supertype of I, J and K will not break the type system. Even if y is properly typed, there is still a problem when an object o matches p rather than q. Both $o.a$ and $o.b$ are legible for the initialization of y with type safety. We take nondeterministic choice here, and any implementation can decide which one to choose in a specific way. With such complexity handling match disjunction, Scala even decides to forbid variable binding in its *pattern alternatives*.

MPU Thesis Template v7.8.1

Chapter 6.

CONCLUSION

We formalize pattern matching for oo languages based on object graphs. Beyond the algebraic view of data objects found in functional languages, and the addition of type tests for oo subtyping, we develop a pattern matching theory tackling the underlying pointer structures of objects, in particular with the intention to examine aliases and cycles. We define the cover relation on layout graphs and the match relation on pattern graphs in terms of graph morphisms, and study the key properties. The theory is further expanded to incorporate match conjunction and disjunction, with more complex cases addressed. We design the language extension in the form of abstract syntax to specify the patterns and match operations. Key algorithms are given in detail to construct the pattern graph, match relation, conjunction and disjunction. We present the type-checking rules and operational semantics for the pattern matching extension. We give the soundness of the theory by proving the type safety theorem. This theory keeps the graphs and subtyping preorder abstract, thus sets a solid and general foundation for oo pattern matching on pointer structures. Together with the discussion on limitations and potential solutions, this work is a reference base for future language design and improvement.

Chapter 7.

ABOUT THIS TEMPLATE

Contents

7.1 Document Class and Options 35
7.2 Commands for Setup 36
7.3 Commands for Display 40
7.4 Emphasis of Author Names in Bibliography 41
7.5 Double-blind Review Format 42
7.6 Preloaded Class and Packages 43
7.7 History 43

7.1 Document Class and Options

This is the MPU Doctoral Thesis Template, Version *7.8.1*, released on Date *2026-05-06*. To use this template, we start a LaTeX document by `\documentclass{mpThesis}`. This document class defaults with options: `Libertine`, `PCOM`, `Proposal`, `Review`, `Screen`, `NoHardback`, `OneHalfSpacing`, and `PhDAfterName`. The available options are categorized as follows.

Font families

- `CM` — The classic Computer Modern family.
- `Times` — The Times Roman family.
- `Utopia` — The Utopia family.
- `Garamond` — The EBGaramond family.
- `Palatino` — The Palatino family.
- † `Libertine` — The Libertine family.

Programmes

- † `PCOM` — Computer Applied Technology.
- `PAIDD` — Artificial Intelligence driven Drug Discovery.
- `PETI` — Educational Technology and Innovation.
- `JUC` — Computer Applied Technology Joint Programme with University of Coimbra.
- `JUB` — Computer Applied Technology Joint Programme with University of Bologna.

Thesis types

† `Proposal` — Research proposals for candidature confirmation.
• `Thesis` — Theses for thesis defences.

Thesis display formats

† `Blind` — Hiding the `BlindRegion` environment and `\BlindText`, and disabling the emphasis of author names, to support the double-blind peer review.
† `Review` — Review format with colored links and vertical rulers.
• `Final` — Final plain format with QR code and defence panel information, no color links and no line numbers.
† `Screen` — Equal inner and outer margins, for displaying on screen.
• `Print` — Larger inner margins. for printing and binding.
† `NoHardback` — No spine text and no calligraphic Chinese MPU watermark.
• `Hardback` — Formal binding format, with spine text and calligraphic Chinese MPU watermark.
† `OneHalfSpacing` — 1.5× line spacing in text.
• `DoubleSpacing` — 2× line spacing in text.
† `PhDAfterName` — "Ph. D." below the student name on the hardcover.
• `NoPhDAfterName` — No "Ph. D." below the student name on the hardcover.

Algorithm packages

- `AlgoMic` — Load the traditional `{algorithm}` and `{algorithmic}` packages.
- `AlgoX` — Load the improved `{algorithm}`, `{algorithmicx}` and `{algpseudocode}` packages.
- `AlgoIIe` — Load the improved and combined `{algorithm2e}` package.
- `MathPar` — Load the `{mathpartir}` package for type setting inference rules.

The `AlgoMic`, `AlgoX` and `AlgoIIe` options are mutually exclusive.

7.2 Commands for Setup

Student and supervisor

- `\StudentName[<break name>]{<full name>}`
  - Purpose: Sets the student's full name and an optional break-friendly version.
  - Arguments:
    * `#1` (optional): Name with line breaks.
    * `#2` (required): Full name of the student.

  - Example: `\StudentName[Brianchon\\Pascal]{Brianchon Pascal}`

- `\StudentNumber{<student number>}`
  - Purpose: Stores the student's identification number.
  - Argument: `#1`: Student number.
  - Example: `\StudentNumber{P-12-1234-1}`

- `\Supervisor{<name>}`
  - Purpose: Sets the name of the supervisor.
  - Argument: `#1`: Name of the supervisor.
  - Example: `\Supervisor{Desargues Monge}`

- `\JointSupervisor{<name>}`
  - Purpose: Sets the name of the joint supervisor.
  - Argument: `#1`: Name of the joint supervisor.
  - Example: `\JointSupervisor{Euler Gauss}`

- `\CoSupervisor{<name>}`
  - Purpose: Sets the name of the co-supervisor, if any.
  - Argument: `#1`: Name of the co-supervisor.
  - Example: `\CoSupervisor{Hilbert Minkowski}`

Date and time

- `\Date{<YYYY-MM-DD>}`
  - Purpose: Sets the document date.
  - Argument: `#1`: Date in `YYYY-MM-DD` format.
  - Example: `\Date{2025-09-17}`

- `\DateToday`
  - Purpose: Sets the document date to the current date.

- `\DefenceDateTime{<YYYY-MM-DD>}{<HH:MM>}`
  - Purpose: Sets the date and time of the thesis defence.
  - Arguments:
    - `#1`: Date in `YYYY-MM-DD` format.
    - `#2`: Time in `HH:MM` format.
  - Example: `\DefenceDateTime{2025-10-01}{14:30}`

Thesis

- `\Title[<break title>]{<full title>}`

  - Purpose: Sets the thesis title and optional break-friendly version.
  - Arguments:
    * `#1` (optional): Title with line breaks.
    * `#2` (required): Title of the thesis.
  - Example: `\Title[AI\\in Healthcare]{AI in Healthcare}`

- `\ThesisNumber{<seq>}{<programme>}{<unit>}{<lna>}{<ftw>}`
  - Purpose: Sets the thesis number components.
  - Arguments:
    * `#1`: Sequence number.
    * `#2`: Programme code.
    * `#3`: Academic unit code.
    * `#4`: Last name of the student.
    * `#5`: First (non-article) word of the title.
  - Example: `\ThesisNumber{00123}{PCOM}{FCA}{pascal}{sample}`

- `\ThesisKeywords{<keywords>}`
  - Purpose: Sets thesis keywords.
  - Argument: `#1`: List of keywords.
  - Example: `\ThesisKeywords{Artificial Intelligence, Healthcare, Machine Learning}`

- `\Keywords{\InsertThesisKeywords}`
  - Purpose: Displays the stored thesis keywords with the “Keywords” prefix.

Chinese Title and Abstract

- `\ChsTitle{<full title>}`
  - Purpose: Sets the thesis title in Simplified Chinese.
  - Arguments: `#1`: Title of the thesis in Simplified Chinese.
  - Example: `\ChsTitle{论文题目}`

- `\ChtTitle{<full title>}`
  - Purpose: Sets the thesis title in Traditional Chinese.

- `\ThesisChsKeywords{<CHS keywords>}`
  - Purpose: Sets thesis keywords in Simplified Chinese.
  - Argument: `#1`: List of keywords in Simplified Chinese.
  - Example: `\ThesisChsKeywords{人工智能、医疗保健、机器学习}`

- `\ThesisChtKeywords{<CHT keywords>}`
  - Purpose: Sets thesis keywords in Traditional Chinese.

- Environment `ChsAbstract`
  - Purpose: Encloses thesis abstract in Simplified Chinese, and displays the abstract with the Chinese thesis title.
  - Example: `\begin{ChsAbstract}`简体中文摘要`\end{ChsAbstract}`
- Environment `ChtAbstract`
  - Purpose: Encloses thesis abstract in Traditional Chinese, and displays the abstract with the Chinese thesis title.
- `\ChsKeywords{\InsertThesisChsKeywords}`
  - Purpose: Displays the stored Simplified Chinese thesis keywords with the "关键字" prefix.
- `\ChtKeywords{\InsertThesisChtKeywords}`
  - Purpose: Displays the stored Traditional Chinese thesis keywords with the "關鍵字" prefix.

Chinese text compiled by pdfLaTeX is implemented by the `CJKutf8` package. There are only two Simplified Chinese fonts built in the TeX Live distribution. If you install your own CJK Chinese fonts in UTF8 encoding, you can specify the fonts to use in the title and abstract, by commands: `\ChsTitleFont`, `\ChtTitleFont`, `\ChsHeaderFont`, `\ChtHeaderFont`, `\ChsTextFont`, and `\ChtTextFont`. However, you don't usually need to touch these commands.

Panel Each of these sets the name of a panel member:

- `\PanelChair{<name>}`
  - Purpose: Sets name of the panel chair.
- `\PanelExternal{<name>}` | `\PanelExternalA{<name>}`
  - Purpose: Sets name of the external examiner.
- `\PanelJointExternal{<name>}` | `\PanelExternalB{<name>}`
  - Purpose: Sets name of the external examiner nominated by the joint university.
- `\PanelInternal{<name>}` | `\PanelInternalA{<name>}` | `\PanelExaminer{<name>}` | `\PanelExaminerA{<name>}`
  - Purpose: Sets name of the internal examiner.
- `\PanelJointInternal{<name>}` | `\PanelInternalB{<name>}` | `\PanelExaminerB{<name>}`
  - Purpose: Sets name of the second internal examiner, or the internal examiner nominated by the joint university.

7.3 Commands for Display

These commands retrieve stored values for use in the document:

Student and supervisor

- `\InsertStudentName`
- `\InsertStudentNameBreak`
- `\InsertStudentNumber`
- `\InsertSupervisor`
- `\InsertJointSupervisor`
- `\InsertCoSupervisor`

University

- `\InsertUniversity`
- `\InsertJointUniversity`
- `\InsertAcademicUnit`
- `\InsertAcademicShort`

Programme

- `\InsertProgramme`
- `\InsertProgrammeBreak`
- `\InsertJointProgramme`
- `\InsertJointProgrammeBreak`
- `\InsertProgrammeCode`

Date and time

- `\InsertDate`
- `\InsertLongDate`
- `\InsertMonthYear`
- `\InsertYear`
- `\InsertDefenceDate`
- `\InsertDefenceTime`

Thesis

- `\InsertTitle`
- `\InsertTitleBreak`
- `\InsertThesisNumber`

- \InsertThesisNumberBreak
- \InsertThesisNumberStyle
- \InsertThesisNumberBreakStyle
- \InsertThesisUrl
- \InsertThesisKeywords
- \InsertThesisChsKeywords

Panel

- \InsertPanelChair
- \InsertPanelExternal
- \InsertPanelJointExternal
- \InsertPanelExternalA
- \InsertPanelExternalB
- \InsertPanelInternal
- \InsertPanelJointInternal
- \InsertPanelInternalA
- \InsertPanelInternalB
- \InsertPanelExaminer
- \InsertPanelExaminerA
- \InsertPanelExaminerB

About this thesis class

- \InsertThesisClassVersion — “7.8.1”
- \InsertThesisClassDate — “2026-05-06”
- \InsertThesisClassDisplayCode — “MPU Thesis Template v7.8.1”

7.4 Emphasis of Author Names in Bibliography

The customized {mpUnsrt.bst} bib-style provides code to scan the author list of a bib-entry to find and highlight the specified author names. The .bst style file is programmed by a postfix language on a stack machine, which is quite interesting to look at.

To specify the author names to emphasize, we need to add entries in the .bib file with category @emphasized. Only two fields are allowed in an entry of this category: author and note. The author field specifies the author list as in a normal category, while the note field specifies the highlighting style, which must be a command taking one argument of the author name, \textbf by default. For example:

```
@emphasized{emphasized/author/list/3,
  author = {Wei Ke and David Hilbert and Gaspard Monge},
```

```
  note    = {\textbf}
}
```

There can be many entries of this category, and one entry must be selected to take effect. To select the entry of the author list, we write `\EmphasizeAuthors{<entry-key>}`. For the above example:

```
\EmphasizeAuthors{emphasized/author/list/3}
```

This must be done before any bib-entries and the inclusion of the `.bst` and `.bib` files.

By the way, we can use command `\bibentry{<entry-key}` to display a bib-entry anywhere in the thesis outside the bibliography list. This is particularly useful for listing the publication produced for the thesis defence, usually in the introduction. See Section 1.3.1 for example.

7.5 Double-blind Review Format

To support the double-blind peer review, option `Blind` hides the student and supervisor names. The option also disables the double degree modes. There are still other sensitive texts that can be used to identify the student and supervisor. They must also be hidden. Such text can be enclosed within environment `BlindRegion`:

```
\chapter{Acknowledgements}\label{ch:ack}

\begin{BlindRegion}
... paragraphs ...
\end{BlindRegion}

\subsection{Current Publication}\label{sec:publication}

\begin{BlindRegion}
\begin{enumerate}[label=(C\arabic*)]
  \item \bibentry{ke2009graph}
\end{enumerate}
\end{BlindRegion}

\chapter{Biography}\label{app:biography}

\begin{BlindRegion}
... paragraphs ...
\end{BlindRegion}
```

Or, such short text can be put in the `\BlindText` macro:

```
#    (c) 2021 \BlindText{Wei Ke and Ka-Hou Chan}
# Id: pp.py 4838 2021-11-14 12:02:46Z \BlindText{wke@IPM.EDU.MO}
```

Then, if option `Blind` is selected, the enclosed text is hidden, otherwise shown as normal. At least

1. the *Acknowledgements* chapter,
2. the publication list, and
3. the *Biography* appendix

must be enclosed in the blind regions. There are also other places that the author information can occur, for example, in the copyright declaration of your implementation code.

While the `Blind` option disables the emphasis of author names in the bibliography, including the `\bibentry` items, the BibTeX may not be able to recompile automatically, you need to remove all the auxiliary files to let the recompilation happen.

7.6 Preloaded Class and Packages

These LaTeX class and packages are loaded by the template:
`[12pt,a4paper]{book}`,
`{geometry}`, `[T1]{fontenc}`, `{CJKutf8}`, `{amsmath,amsthm,amssymb}`, `{mathtools}`,
`{makecell}`, `{array}`, `{bm}`,
`[dvipsnames,x11names,svgnames]{xcolor}`, `[pdftex]{hyperref}`,
`{multirow,multicol}`, `{wrapfig}`, `{graphicx}`, `{caption,subcaption}`,
`[style=long4col,toc,automake]{glossaries}`, `[noindentafter]{titlesec}`, `{titletoc}`,
`{tocbibind}`, `{notoccite}`, `{setspace}`, `{fancyhdr}`, `{etoolbox}`, `{stmaryrd}`, `{marvosym}`,
`[en-GB,calc]{datetime2}`, `{varwidth}`, `{keyval}`, `{qrcode}`, and `{bibentry}`.

A few packages are optionally loaded:
`{eso-pic}` (option `Hardback`), `{fgruler}` (option `Review`), and `{comment}` (option `Blind`).

7.7 History

v7.8.1 • 6th May 2026

- Fix a typo;

v7.8 • 20th April 2026

- Add Traditional Chinese title/abstract/keyword support:
  `\ChtTitle`, `\ThesisChtKeywords`, `\ChtKeywords`, and environment `ChtAbstract`;

v7.7 • 27th March 2026

- Replace line numbers with side rulers;
- Add the `Blind` option, together with `BlindRegion` and `\BlindText` to support double-blind peer review;
- Add the `\CoSupervisor` and `\InsertCoSupervisor` commands to set and display the official co-supervisor of a student, if any;
- Add macros about this thesis class — version number, release date, and display code;

v7.6 • 6th March 2026

- Add Simplified Chinese title/abstract/keyword support:
  `\ChsTitle`, `\ThesisChsKeywords`, `\ChsKeywords`, and environment `ChsAbstract`;
- Add a thesis skeleton, with brief descriptions of what are expected in the individual chapters;

v7.5 • 10th October 2025

- fix the missing package bug with the `Hardback` option;
- Pack all logos into one PDF file and use page selection in `\includegraphics` to load individual logos;
- Add a research proposal skeleton, with brief descriptions of what are expected in the individual chapters;

v7.4 • 30th September 2025

- options to include and integrate various algorithm packages: `algorithm`, `algorithmic`, `algorithmicx`, `algpseudocode`, `algorithm2e` and `mathpartir`;
- new `sample-algo.tex` to test the included algorithm packages;

v7.3 • 18th September 2025

- new (`Review` mode) class option `Screen` (opposite: `Print`) to turn off odd/even page margin difference, better for screen display;
- new sample thesis modified from a real published paper to show more LaTeX type setting examples;
- restart line numbers from 1 for every page;

- add a chapter to list the document class options and the commands provided;
- move the version history to the above chapter;
- fix some `lineno` issues in display math, and extend line numbers to appendices;

v7.2 • 13th August 2025

- new set of panel member definitions for joint-degree programmes,
  `\PanelChair`
  `\PanelExternal` or `\PanelExternalA`
  `\PanelJointExternal` or `\PanelExternalB`
  `\PanelInternal` or `\PanelInternalA` or `\PanelExaminer` or `\PanelExaminerA`
  `\PanelJointInternal` or `\PanelInternalB` or `\PanelExaminerB`;
- fine tune the logos for joint programme;
- adjust signature page;
- remove the MPI glossary;

v7.1 • 19th June 2025

- add `\PanelExaminerC` and `\PanelExaminerD` for joint programme;
- adjust signature page;

v7.0 • 16th June 2025

- add `\InsertUniversity` and `\InsertJointUniversity`;
- add `\JointSupervisor` for use with joint programme;
- adjust the font size and rewrite the statement from the cover page;
- refine “mpThesis.cls”;

v6.4 • 23rd May 2025

- provide descriptions and examples for various options of “mpThesis”;
- address the code for entering long name of author/title/programme;
- Co-supervisor will not be showed;
- add `\InsertAcademicShort` for the use of ThesisNumber;
- add `\InsertProgrammeCode` for the use of ThesisNumber;

v6.3 • 21st May 2025

- add options of `PCOM`, `PAIDD`, `PETI` for Ph.D. programmes;
- add option of `JUC`, `JUB` for joint programmes;
- resize/adjust the MPU logo(s) in FrontMatter;
- providing the UC logo for the use of FCA-JUC programme;

- rewrite the definition of “font families” in .cls;
- rewrite the definition of “thesis types” in .cls;
- rewrite the definition of “produce modes” in .cls;
- rewrite the of **ACKNOWLEDGMENTS**;

v6.2 • 25th March 2024

- enable “x11names” from package “xcolor”;
- redraw MPU-FCA-logo.pdf;

v6.1 • 3rd April 2025

- use CalliCover as the default `Hardback` paper;

v6.0 • 24th March 2023

- add `Hardback` option for final printing;
- add “copyright page” in between “cover page” and “title page”;
- redesign “cover page”, “title page”, and “declaration page”;
- “declaration page” is created as a *.tex file and is allowed to be modified;
- `\DefenceDate`, `\PanelChair`, `\PanelExternal` and `\PanelExaminer` information needs to be entered;
- supports optional long title names (with line break) to be displayed on the copyright page only;
- a template of “Author Biography” is provided;
- make font “Libertine” and cover “CalliCover” the default;
- move “Ethics Checklist” to appendix;
- latex (macros) code refinement;

v5.9 • 6th January 2023

- support “subsubsection” indexing;
- “notoccite” package is enabled by default in “mpThesis.cls”;

v5.8 • 4th January 2023

- a blank page has been added at the end of thesis;
- fixed issue from package “glossaries”;
- fixed template mark displaying in “Final” mode, now only on the back of the cover page;
- added the `DoubleSpacing` option to enable double spacing when needed, the default is one-half spacing;
- added the `NoTocCite` option to include the “notoccite” package to prevent the numbering of bibliography from being affected by the citations in TOC;

v5.7 • 27th October 2022

- restrict package “glossaries” using version 4.46;
- define the (MPU and MPI glossary by default to eliminate the quirk in the “glossaries” package;
- determine double-spacing in mainMatter;
- increase TOC depth level to 2;
- show the “url” and “doi” items in the “mpUnsrt” style.
- a sample of “publication list” is provided in Chapter 1;
- section refinement in Chapter 1 to Chapter 3;
- adjust “citecolor”;

v5.6 • 22nd September 2022

- made “List of Figures/Tables/Algorithms/Abbreviations” optional;
- adjusted the position of the example section;
- provided more advanced examples;
- reformatted section “Template Versioning”;
- release to canvas;

v5.5 • 20th September 2022

- added “List of Algorithms”;
- provided an example of algorithm in Chapter 4;
- added optional long name (with line breaks) support, including student names and programme names, shown only on the title page;
- `NoToC` option;
- adjusted the vertical position of the Chinese calligraphic MPU logo in page headers;
- adjusted vertical spacing of the table on the title page;

v5.4 • 15th September 2022

- refined title page;
- adjusted spacing;
- added Chinese MPU logo (in .pdf);
- redesigned header format;

v5.3 • 13th September 2022

- refined format of “mpUnsrt.bst”;
- added emphasized authors support to “mpUnsrt.bst”;
- improved TOC macros;

v5.2 • 6th September 2022

- added reference style “mpUnsrt.bst”;
- added more font families;
- disposed “IndentFirst”;
- improved template manuals;
- make minitoc singlespacing;
- provided two types of `Thesis` or `Proposal`, `Thesis` is default;
- provided two modes of `Review` or `Final`, `Review` is default;
- `Review` mode shows hyperlinks and line numbers;
- `Final` mode removes line numbers, no color for hyperlinks;
- minor font size adjustment;
- added template version marks to even pages;

v5.1 • 2nd September 2022

- Overleaf online tested;
- online version refinement;
- made “IndentFirst” optional;
- redefined the cross-reference prefix;
- aligned list of abbreviations;

v5.0 • 29th August 2022

- major revision;
- item definition of template classes and refinement of LaTeX code;
- regeneration of MPU logo;
- provide more font family;
- an example about profile updating is provided;

v4.8 • 24th August 2022

- add mini-tables-of-contents in each chapter;
- move “Acknowledgments” to frontmatter;
- add “List of Abbreviations”;
- provide an example of adding glossary in Chapter 3;
- add the section of “Template Versioning” in Chapter 6;
- the first indent have been changed to default;
- latex (macros) code refinement;

v4.2 • 8th March 2022

- add “Universidade Politécnica de Macau” on the Cover page;
- cover page redesign;
- regenerate the optimized MPU loge;
- refine TOC format;

v4.1 • 4th March 2022

- appendix header correction;
- section header alignment;

v4.0 • 7th March 2022

- update “MPI” to “MPU”;
- update “School” to “Faculty”;
- adjust the size and position of the MPU logo.
- formatting student ID: `P-XX-XXXX-X`;
- change to use small cap format for the section header;
- highlight fillable item;
- few formatting adjustments;

v3.7 • 26th November 2021

- dispose the front of “Times New Roman”;
- update title page;
- provide the logo in PDF format;
- few formatting adjustments;

v3.5 • 13th April 2021

- update TOC’s chapter and title;
- update the chapter of “Acknowledgements”;
- provide an example of using equation in Chapter 1;
- provide an example of using table in Chapter 2;
- provide an example of adding figure in Chapter 3;
- provide an example of using list in Chapter 5;
- provide an example of citing literature in Chapter 5;
- other minor adjust to format;

v3.1 • 27th January 2021

- update cover paper statement;
- change citation style to ieeetr style;
- remove chinese abstract;
- few formatting adjustments;

v3.0 • 15th January 2021

- format TOC;
- change first indent to 1.2cm;
- change header/footer font size to 10;
- other adjustments;

v2.3 • 7th January 2021

- supported chinese abstract;
- adjust TOC;

v2.1 • 6th January 2021

- update cover pager;
- design and formatting;

v1.0 • 4th January 2021

- first release;

BIBLIOGRAPHY

[1] F. Warren Burton and Robert D Cameron. "Pattern matching with abstract data types". *Journal of Functional Programming*, 3(2):171–190, 1993. (https://doi.org/10.1017/S095679680000068X).

[2] Martin Odersky, Philippe Altherr, Vincent Cremet, Gilles Dubochet, Burak Emir, Philipp Haller, Stéphane Micheloud, Nikolay Mihaylov, Adriaan Moors, Lukas Rytz, Michel Schinz, Erik Stenman, and Matthias Zenger. "Scala Language Specification Version 2.13", 2021. Accessed: 2021-09-30, (https://scala-lang.org/files/archive/spec/2.13/).

[3] Wei Ke, Zhiming Liu, Shuling Wang, and Liang Zhao. "A graph-based operational semantics of OO programs". In *International Conference on Formal Engineering Methods*, pages 347–366. Springer, 2009.

[4] Charles AR Hoare and He Jifeng. "A trace model for pointers and objects". In *European Conference on Object-Oriented Programming*, pages 1–18. Springer, 1999.

[5] Martin Abadi and Luca Cardelli. *A Theory of Objects*. Springer, 1996.

[6] Barbara H Liskov and Jeannette M Wing. "A behavioral notion of subtyping". *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 16(6):1811–1841, 1994.

[7] Felix Geller, Robert Hirschfeld, and Gilad Bracha. *Pattern Matching for an object-oriented and dynamically typed programming language*. Number 36. Universitätsverlag Potsdam, 2010.

[8] Baltasar Trancón y Widemann and Markus Lepper. "Paisley: Pattern matching à la carte". In *International Conference on Theory and Practice of Model Transformations*, pages 240–247. Springer, 2012.

[9] Yuriy Solodkyy, Gabriel Dos Reis, and Bjarne Stroustrup. "Open pattern matching for C++". In *Proceedings of the 12th international conference on Generative programming: concepts & experiences*, pages 33–42, 2013.

[10] Sukyoung Ryu, Changhee Park, and Guy L Steele Jr. "Adding pattern matching to existing object-oriented languages". In *ACM SIGPLAN Foundations of Object-Oriented Languages Workshop*, volume 5. Citeseer, 2010.

[11] Radu Muschevici, Alex Potanin, Ewan Tempero, and James Noble. "Multiple dispatch in practice". *Acm sigplan notices*, 43(10):563–582, 2008.

[12] Craig Chambers. "Object-oriented multi-methods in Cecil". In *European Conference on Object-Oriented Programming*, pages 33–56. Springer, 1992.

[13] Michael Ernst, Craig Kaplan, and Craig Chambers. "Predicate dispatching: A unified theory of dispatch". In *European Conference on Object-Oriented Programming*, pages 186–211. Springer, 1998.

[14] Curtis Clifton, Todd Millstein, Gary T Leavens, and Craig Chambers. "MultiJava: Design rationale, compiler implementation, and applications". *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 28(3):517–575, 2006.

[15] Jed Liu and Andrew C. Myers. "JMatch: Iterable Abstract Pattern Matching for Java". In Veronica Dahl and Philip Wadler, editors, *Practical Aspects of Declarative Languages*, pages 110–127, Berlin, Heidelberg, 2003. Springer Berlin Heidelberg.

[16] Burak Emir, Martin Odersky, and John Williams. "Matching objects with patterns". In *European Conference on Object-Oriented Programming*, pages 273–298. Springer, 2007.

[17] Tobias Kohn, Guido van Rossum, Gary Brandt Bucher II, and Ivan Levkivskyi. "Dynamic pattern matching with Python". In *Proceedings of the 16th ACM SIGPLAN International Symposium on Dynamic Languages*, pages 85–98, 2020.

[18] Brandt Bucher and Guido van Rossum. "PEP 634 – structural pattern matching: Specification", Sep 2020. Accessed: 2021-09-30, (https://www.python.org/dev/peps/pep-0634/).

[19] Tobias Kohn and Guido van Rossum. "PEP 635 – structural pattern matching: Motivation and rationale", Sep 2020. Accessed: 2021-09-30, (https://www.python.org/dev/peps/pep-0635/).

[20] Gavin Bierman. "JEP 394: Pattern Matching for instanceof", Jul 2020. Accessed: 2021-09-30, (https://openjdk.java.net/jeps/394/).

[21] Microsoft. "What's new in C# 8.0", Jul 2020. Accessed: 2021-09-30, (https://docs.microsoft.com/en-us/dotnet/csharp/whats-new/csharp-8).

[22] Carl A Gunter. *Semantics of programming languages: structures and techniques*. MIT press, 1992.

[23] Richard Bornat. "Proving Pointer Programs in Hoare Logic". In Roland Backhouse and José Nuno Oliveira, editors, *Mathematics of Program Construction*, pages 102–126, Berlin, Heidelberg, 2000. Springer Berlin Heidelberg.

MPU Thesis Template v7.8.1

[24] Wei Ke, Zhiming Liu, Shuling Wang, and Liang Zhao. "A graph-based generic type system for object-oriented programs". *Frontiers of Computer Science*, 7(1):109–134, 2013.

[25] Liang Zhao, Xiaojian Liu, Zhiming Liu, and Zongyan Qiu. "Graph transformations for object-oriented refinement". *Formal Aspects of Computing*, 21(1):103–131, 2009.

[26] Liang Zhao, Shuling Wang, and Zhiming Liu. "Graph-based object-oriented Hoare logic". In *Theories of Programming and Formal Methods*, pages 374–393. Springer, 2013.

[27] Nadime Francis, Alastair Green, Paolo Guagliardo, Leonid Libkin, Tobias Lindaaker, Victor Marsault, Stefan Plantikow, Mats Rydberg, Petra Selmer, and Andrés Taylor. "Cypher: An Evolving Query Language for Property Graphs". In *Proceedings of the 2018 International Conference on Management of Data*, SIGMOD '18, page 1433–1445, New York, NY, USA, 2018. Association for Computing Machinery. (https://doi.org/10.1145/3183713.3190657).

[28] Qiang Tong. "Mapping object-oriented database models into RDF (S)". *IEEE Access*, 6:47125–47130, 2018.

[29] Jia Li, Yang Cao, and Shuai Ma. "Relaxing Graph Pattern Matching With Explanations". In *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management*, CIKM '17, page 1677–1686, New York, NY, USA, 2017. Association for Computing Machinery. (https://doi.org/10.1145/3132847.3132992).

[30] Silvia Ghilezan. "Strong normalization and typability with intersection types". *Notre Dame Journal of Formal Logic*, 37(1):44–52, 1996.

[31] ISO/IEC. *Working Draft, Standard for Programming Language C++*. Number n4950. International Organization for Standardization (ISO), Geneva, Switzerland, 2023. (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/n4950.pdf).

Appendix A.

IMPLEMENTATION SOURCE CODE

Copyright © 2021 Wei Ke and Ka-Hou Chan.

The type-checking and operational semantics of the pattern matching described in the thesis, as well as the test cases, are implemented in Python. The source code listed is licensed under GPL-3, see the license file in Appendix B.

A.1 Abstract syntax of this minimal oo language, except pattern matching

asx.py

```
# (c) 2021
# license: GPL-3
#
from collections import namedtuple

PrintStmt = namedtuple('PrintStmt', ['args'])
AssignStmt = namedtuple('AssignStmt', ['lexpr', 'expr'])
IfStmt = namedtuple('IfStmt', ['expr', 'then_stmt', 'else_stmt'])
WhileStmt = namedtuple('WhileStmt', ['expr', 'stmt'])
BlockStmt = namedtuple('BlockStmt', ['stmts'])

VarDecl = namedtuple('VarDecl', ['label', 'cla'])
VarEnd = namedtuple('VarEnd', ['label'])

VarExpr = namedtuple('VarExpr', ['label'])
AttrExpr = namedtuple('AttrExpr', ['expr', 'label'])
OpExpr = namedtuple('OpExpr', ['op', 'args'])
NewExpr = namedtuple('NewExpr', ['cla'])
AndExpr = namedtuple('AndExpr', ['left', 'right'])
OrExpr = namedtuple('OrExpr', ['left', 'right'])

Program = namedtuple('Program', ['block'])
#
# end of asx.py
# Id: asx.py 4842 2021-11-14 17:09:57Z
```

A.2 Abstract syntax of this pattern matching

pattern.py

```
# (c) 2021
# license: GPL-3
#
from collections import namedtuple
```

```

LabeledPattern = namedtuple('LabeledPattern', ['label', 'base'])
PatternRef = namedtuple('PatternRef', ['label'])
ClassPattern = namedtuple('ClassPattern', ['cla', 'attrs'])
PatternConj = namedtuple('PatternConj', ['patterns'])
PatternDisj = namedtuple('PatternDisj', ['patterns'])
Case = namedtuple('Case', ['junc', 'stmt', 'extra'])
MatchStmt = namedtuple('MatchStmt', ['expr', 'cas'])

class Extra:
    def __init__(self, extra = None):
        self.extra = extra

    def put(self, extra):
        self.extra = extra

    def get(self):
        return self.extra
#
# end of pattern.py
# Id: pattern.py 4838 2021-11-14 12:02:46Z
```

A.3 Prety-printing of dictionaries

pp.py

```
# (c) 2021
# license: GPL-3
#
from collections import namedtuple

PrfxPP = namedtuple('PrefixedPP', ['prefix', 'pp'])

def pprint(s, name = None):
    def pp(s, indent):
        if isinstance(s, dict):
            if s:
                print('{')
                for k, v in s.items():
                    indent2 = indent+4
                    print('{}{!r}:␣'.format('␣'*indent2, k), end = '')
                    pp(v, indent2)
                print('␣'*indent+'}')
            else:
                print('{}')
        else:
            print('{!r}'.format(s))

    if type(s) is PrfxPP:
        p, t = s
        if p:
            if name:
                print('{}␣{}:␣'.format(p, name), end = '')
            else:
                print('{}:␣'.format(p), end = '')
        pp(t, 0)
    else:
        if name:
            print('{}:␣'.format(name), end = '')
```

```
34          pp(s, 0)
35  #
36  # end of pp.py
37  # Id: pp.py 4838 2021-11-14 12:02:46Z
```

A.4 Bijective dictionary

bidict.py

```
1   # (c) 2021
2   # license: GPL-3
3   #
4   class BiDictError(Exception): pass
5   class UniqueInvError(BiDictError): pass
6   class InterInvError(BiDictError): pass
7
8   class bidict(dict):
9       def __init__(self, *args, **kwargs):
10          super().__init__(*args, **kwargs)
11          self.inv = {}
12          for k, v in self.items():
13              self.inv.setdefault(v, []).append(k)
14
15      def __setitem__(self, k, v):
16          if k in self:
17              x = self[k]
18              self.inv[x].remove(k)
19              if not self.inv[x]:
20                  del self.inv[x]
21          super().__setitem__(k, v)
22          self.inv.setdefault(v, []).append(k)
23
24      def __delitem__(self, k):
25          v = self[k]
26          self.inv.setdefault(v, []).remove(k)
27          if not self.inv[v]:
28              del self.inv[v]
29          super().__delitem__(k)
30
31      def unique_inv(self, v):
32          ks = self.inv[v]
33          if len(ks) != 1:
34              raise UniqueInvError()
35          return ks[0]
36
37
38  def dict_union(ds):
39      return {k: v for d in ds for k, v in d.items()}
40
41  def bidict_union(bds):
42      return bidict(dict_union(bds))
43  #
44  # end of bidict.py
45  # Id: bidict.py 4838 2021-11-14 12:02:46Z
```

A.5 Graphs and graph algorithms

graph.py

```
# (c) 2021
# license: GPL-3
#
from collections import namedtuple
from queue import Queue

from bidict import bidict
from pp import PrfxPP

from subtype import NULL_TYPE, ValueSet, exists_ty_le_all, ty_inf, ty_sup

from pattern import LabeledPattern, PatternRef, ClassPattern

LayoutGraph = namedtuple('LayoutGraph', ['nodes', 'edges', 'root'])
PatternGraph = namedtuple('PatternGraph', ['layout', 'types'])
StateGraph = namedtuple('StateGraph', ['layout', 'types', 'values'])
Node = namedtuple('Node', ['id'])
Label = namedtuple('Label', ['id'])
Edges = namedtuple('Edges', ['labels', 'targets'])

class GraphError(Exception): pass
class Mismatch(GraphError):  pass
class NoUnion(GraphError):  pass
class NoConj(GraphError):  pass
class UndefRef(GraphError):  pass
class RedefRef(GraphError):  pass
class NoScope(GraphError):  pass
class UndefVar(GraphError):  pass
class UndefAttr(GraphError):  pass

def unzip2(s):
    if not s:
        return ([], [])
    else:
        return list(zip(*s))

def unzip4(s):
    if not s:
        return ([], [], [], [])
    else:
        return list(zip(*s))

def set_union(ss):
    return set().union(*ss)

def set_inter(ss):
    i = iter(ss)
    for s in i:
        return set(s).intersection(*i)

    return set()

def layout_graph_to_pp(g):
    ns, es, p = g
    pp = {}
    pp['root'] = '({})'.format(p.id)
    pp['nodes'] = sorted(u.id for u in ns)
```

```
    pp['edges'] = {u.id: [(la.id, es.targets[(u, la)].id) for la in es.labels[u]]
                   for u in sorted(ns)}
    return PrfxPP('LayoutGraph', pp)

class NodeFactory:
    def __init__(self):
        self._cur_id = 0

    def new_node(self):
        self._cur_id += 1
        return Node(self._cur_id)

NODE_FACTORY = NodeFactory()

def new_node():
    return NODE_FACTORY.new_node()

def init_state_graph():
    p = new_node()
    return StateGraph(LayoutGraph({p}, Edges({p: set()}, {}), p), {}, {})

def gc_layout(g):
    ns, es, r = g
    queue = Queue()
    queue.put(r)
    ns2 = set()
    es2 = Edges(dict(), dict())

    while not queue.empty():
        p = queue.get()
        if p not in ns2:
            ns2.add(p)
            las = set(es.labels[p])
            es2.labels[p] = las
            for la in las:
                q = es.targets[(p, la)]
                es2.targets[(p, la)] = q
                queue.put(q)

    return LayoutGraph(ns2, es2, r)

def gc_state(sg):
    g, tm, vm = sg
    g2 = gc_layout(g)
    tm2 = {p: t for (p, t) in tm.items() if p in g2.nodes}
    vm2 = {p: v for (p, v) in vm.items() if p in g2.nodes}
    return StateGraph(g2, tm2, vm2)

def extract_pattern(sg, p):
    (ns, es, r), tm, vm = sg
    g2 = gc_layout(LayoutGraph(ns, es, p))
    tm2 = {u: ValueSet({vm[u]}) if u in vm else tm[u] for u in g2.nodes}
    return PatternGraph(g2, tm2)

def swing_layout(g, p, la, q):
    ns, es, r = g
    es.labels[p].add(la)
    es.targets[(p, la)] = q

def swing_state(sg, p, la, q):
    swing_layout(sg.layout, p, la, q)

```

```
120 def add_object_to_layout(g, cla):
121     ns, es, r = g
122     p = new_node()
123     ns.add(p)
124     las = set(cla.attrs)
125     es.labels[p] = las
126     qs = []
127     for la in las:
128         q = new_node()
129         ns.add(q)
130         es.labels[q] = set()
131         es.targets[(p, la)] = q
132         qs.append(q)
133 
134     return (p, qs)
135 
136 def add_object_to_state(sg, cla):
137     g, tm, vm = sg
138     p, qs  = add_object_to_layout(g, cla)
139     tm[p] = cla
140     for q in qs:
141         tm[q] = NULL_TYPE
142 
143     return p
144 
145 def add_value_to_state(sg, v):
146     p  = add_object_to_state(sg, v.cla)
147     sg.values[p] = v
148     return p
149 
150 def push_state(sg, sla):
151     (ns, es, r), tm, vm = sg
152     r2 = new_node()
153     ns.add(r2)
154     es.labels[r2] = {sla}
155     es.targets[(r2, sla)] = r
156     return StateGraph(LayoutGraph(ns, es, r2), tm, vm)
157 
158 def pop_state(sg, sla):
159     (ns, es, r), tm, vm = sg
160     if sla not in es.labels[r]:
161         raise NoScope()
162 
163     return gc_state(StateGraph(LayoutGraph(ns, es, es.targets[(r, sla)]), tm, vm))
164 
165 def find_lvar(sg, sla, la):
166     (ns, es, r), tm, vm = sg
167     while la not in es.labels[r] and sla in es.labels[r]:
168         r = es.targets[(r, sla)]
169 
170     if la not in es.labels[r]:
171         raise UndefVar()
172 
173     return (r, la)
174 
175 def find_var(sg, sla, la):
176     return sg.layout.edges.targets[find_lvar(sg, sla, la)]
177 
178 def find_lattr(sg, p, la):
179     (ns, es, r), tm, vm = sg
180     if not p in ns:
181         raise UndefVar()
```

```
182
183     if la not in es.labels[p]:
184         raise UndefAttr()
185
186     return (p, la)
187
188 def find_attr(sg, p, la):
189     return sg.layout.edges.targets[find_lattr(sg, p, la)]
190
191 def cons_match(g1, g2, le, tau1, tau2):
192     ns1, es1, p1 = g1
193     ns2, es2, p2 = g2
194     f = bidict()
195
196     def dfs_match(es1, p1, es2, p2):
197         if p1 in f:
198             if f[p1] != p2:
199                 raise Mismatch()
200         else:
201             if not le(tau2[p2], tau1[p1]):
202                 raise Mismatch()
203             if p2 in f.inv:
204                 raise Mismatch()
205
206             f[p1] = p2
207
208             for la in es1.labels[p1]:
209                 q1 = es1.targets[(p1, la)]
210                 if la not in es2.labels[p2]:
211                     raise Mismatch()
212                 q2 = es2.targets[(p2, la)]
213                 dfs_match(es1, q1, es2, q2)
214
215     dfs_match(es1, p1, es2, p2)
216     return f
217
218 def cons_union(gs):
219     ess, ps = list(zip(*((es, p) for ns, es, p in gs)))
220     fs = [bidict() for g in gs]
221
222     def dfs_union(iz, ps):
223         kz = [k for k in iz if ps[k] in fs[k]]
224         cz = [c for c in iz if ps[c] not in fs[c]]
225         uz = [fs[k][ps[k]] for k in kz]
226         if len(uz) > 1:
227             raise NoUnion()
228         if cz:
229             if uz:
230                 pd = uz[0]
231                 if any(pd in fs[c].inv for c in cz):
232                     raise NoUnion()
233             else:
234                 pd = new_node()
235             for c in cz:
236                 fs[c][ps[c]] = pd
237             laz = set_union(ess[i].labels[ps[i]] for i in iz)
238             for la in laz:
239                 qs = [None]*len(gs)
240                 jz = [j for j in iz if la in ess[j].labels[ps[j]]]
241                 for j in jz:
242                     qs[j] = ess[j].targets[(ps[j], la)]
243                 dfs_union(jz, qs)
```

```
244
245     dfs_union(list(range(len(gs))), ps)
246     return fs
247
248 def cons_inter(gs):
249     ess, ps = unzip2((es, p) for ns, es, p in gs)
250     fs = [bidict() for g in gs]
251
252     def dfs_inter(ps):
253         uz = [f[p] for f, p in zip(fs, ps) if p in f]
254         if uz:
255             pc = uz[0]
256         else:
257             pc = new_node()
258         for f, p in zip(fs, ps):
259             f[p] = pc
260         if len(uz) != len(ps):
261             laz = set_inter(es.labels[p] for es, p in zip(ess, ps))
262             for la in laz:
263                 qs = [es.targets[(p, la)] for es, p in zip(ess, ps)]
264                 dfs_inter(qs)
265
266     dfs_inter(ps)
267     return fs
268
269 def cons_match_conj(pgs):
270     gs, tms = unzip2(pgs)
271     fs = cons_union(gs)
272     tsd = {}
273     nsd = set_union(f.inv for f in fs)
274     for ud in nsd:
275         ts = {tm[f.unique_inv(ud)] for tm, f in zip(tms, fs) if ud in f.inv}
276         if not exists_ty_le_all(ts):
277             raise NoConj()
278         td = ty_inf(ts)
279         tsd[ud] = td
280
281     return (fs, tsd)
282
283 def cons_match_disj(pgs):
284     gs, tms = unzip2(pgs)
285     fs = cons_inter(gs)
286     tsc = {}
287     nsc = set_union(f.inv for f in fs)
288     for uc in nsc:
289         ts = set_union({tm[u] for u in f.inv[uc]} for tm, f in zip(tms, fs))
290         tc = ty_sup(ts)
291         tsc[uc] = tc
292
293     return (fs, tsc)
294
295 def cons_pattern_graph(pattern):
296     ns = set()
297     es = Edges({}, {})
298     tm = {}
299     rm = bidict()
300
301     def parse(pattern):
302         if type(pattern) is LabeledPattern:
303             la = pattern.label
304             p = parse(pattern.base)
305             if la in rm:
```

```
                r = rm[la]
                if r in ns:
                    raise RedefRef()
                'E[p/r]'
                nlas = [nla for nla, u in es.targets.items() if u == r]
                for nla in nlas:
                    es.targets[nla] = p
                'R[p/r]'
                las2 = [la2 for la2, u in rm.items() if u == r]
                for la2 in las2:
                    rm[la2] = p

            rm[la] = p
        elif type(pattern) is ClassPattern:
            cla, attrs = pattern
            p = new_node()
            ns.add(p)
            tm[p] = cla
            las, qs = unzip2([(la, parse(pattern2))
                              for la, pattern2 in attrs.items()])
            es.labels[p] = set(las)
            for la, q in zip(las, qs):
                es.targets[(p, la)] = q
        elif type(pattern) is PatternRef:
            la = pattern.label
            if la in rm:
                p = rm[la]
            else:
                p = new_node()
                rm[la] = p
        elif type(pattern) is ValueSet:
            p = new_node()
            ns.add(p)
            es.labels[p] = set()
            tm[p] = pattern

        return p

    p = parse(pattern)
    if any(u not in ns for u in rm.inv):
        raise UndefRef()

    return (LayoutGraph(ns, es, p), tm, rm)
#
# end of graph.py
# Id: graph.py 4838 2021-11-14 12:02:46Z
```

A.6 Operations on values

op.py

```
# (c) 2021
# license: GPL-3
#
from collections import namedtuple

import operator

from subtype import Value, INT_TYPE, STR_TYPE, BOOL_TYPE
```

```

from graph import Label, add_value_to_state

OpDef = namedtuple('OpDef', ['op', 'par_types', 'res_type', 'f'])

OP_TAB = {}

def is_op(la):
    return la in OP_TAB

def get_op(la):
    return OP_TAB[la]

def invoke_op(sg, op, args):
    f = OP_TAB[op].f
    return f(sg, *args)

def op_binary(sg, x, y, t, f):
    g, tm, vm = sg
    return add_value_to_state(sg, Value(t, f(vm[x].value, vm[y].value)))

def op_unary(sg, x, t, f):
    g, ts, vs = sg
    return add_value_to_state(sg, t, Value(t, f(vs[x].value)))

def op_add(sg, x, y):
    return op_binary(sg, x, y, INT_TYPE, operator.add)

def op_sub(sg, x, y):
    return op_binary(sg, x, y, INT_TYPE, operator.sub)

def op_mul(sg, x, y):
    return op_binary(sg, x, y, INT_TYPE, operator.mul)

def op_div(sg, x, y):
    return op_binary(sg, x, y, INT_TYPE, operator.floordiv)

def op_mod(sg, x, y):
    return op_binary(sg, x, y, INT_TYPE, operator.mod)

def op_neg(sg, x):
    return op_unary(sg, x, INT_TYPE, operator.neg)

def op_not(sg, x):
    return op_unary(sg, x, BOOL_TYPE, operator.not_)

def op_cat(sg, x, y):
    return op_binary(sg, x, y, STR_TYPE, operator.concat)

def op_lower(sg, x):
    return op_unary(sg, x, STR_TYPE, lambda x: x.lower())

def op_upper(sg, x):
    return op_unary(sg, x, STR_TYPE, lambda x: x.lower())

def op_ieq(sg, x, y):
    return op_binary(sg, x, y, BOOL_TYPE, operator.eq)

def op_ine(sg, x, y):
    return op_binary(sg, x, y, BOOL_TYPE, operator.ne)

def op_ilt(sg, x, y):
```

```
71      return op_binary(sg, x, y, BOOL_TYPE, operator.lt)
72
73  def op_ile(sg, x, y):
74      return op_binary(sg, x, y, BOOL_TYPE, operator.le)
75
76  def op_igt(sg, x, y):
77      return op_binary(sg, x, y, BOOL_TYPE, operator.gt)
78
79  def op_ige(sg, x, y):
80      return op_binary(sg, x, y, BOOL_TYPE, operator.ge)
81
82  def op_seq(sg, x, y):
83      return op_binary(sg, x, y, BOOL_TYPE, operator.eq)
84
85  def op_sne(sg, x, y):
86      return op_binary(sg, x, y, BOOL_TYPE, operator.ne)
87
88  def op_slt(sg, x, y):
89      return op_binary(sg, x, y, BOOL_TYPE, operator.lt)
90
91  def op_sle(sg, x, y):
92      return op_binary(sg, x, y, BOOL_TYPE, operator.le)
93
94  def op_sgt(sg, x, y):
95      return op_binary(sg, x, y, BOOL_TYPE, operator.gt)
96
97  def op_sge(sg, x, y):
98      return op_binary(sg, x, y, BOOL_TYPE, operator.ge)
99
100 OP_TAB = {
101     Label('add'): OpDef(Label('add'), [INT_TYPE, INT_TYPE], INT_TYPE, op_add),
102     Label('sub'): OpDef(Label('sub'), [INT_TYPE, INT_TYPE], INT_TYPE, op_sub),
103     Label('mul'): OpDef(Label('mul'), [INT_TYPE, INT_TYPE], INT_TYPE, op_mul),
104     Label('div'): OpDef(Label('div'), [INT_TYPE, INT_TYPE], INT_TYPE, op_div),
105     Label('mod'): OpDef(Label('mod'), [INT_TYPE, INT_TYPE], INT_TYPE, op_mod),
106     Label('neg'): OpDef(Label('neg'), [INT_TYPE], INT_TYPE, op_neg),
107     Label('not'): OpDef(Label('not'), [BOOL_TYPE], BOOL_TYPE, op_not),
108     Label('cat'): OpDef(Label('cat'), [STR_TYPE, STR_TYPE], STR_TYPE, op_cat),
109     Label('lower'): OpDef(Label('lower'), [STR_TYPE], STR_TYPE, op_lower),
110     Label('upper'): OpDef(Label('upper'), [STR_TYPE], STR_TYPE, op_upper),
111     Label('ieq'): OpDef(Label('ieq'), [INT_TYPE, INT_TYPE], BOOL_TYPE, op_ieq),
112     Label('ine'): OpDef(Label('ine'), [INT_TYPE, INT_TYPE], BOOL_TYPE, op_ine),
113     Label('ilt'): OpDef(Label('ilt'), [INT_TYPE, INT_TYPE], BOOL_TYPE, op_ilt),
114     Label('ile'): OpDef(Label('ile'), [INT_TYPE, INT_TYPE], BOOL_TYPE, op_ile),
115     Label('igt'): OpDef(Label('igt'), [INT_TYPE, INT_TYPE], BOOL_TYPE, op_igt),
116     Label('ige'): OpDef(Label('ige'), [INT_TYPE, INT_TYPE], BOOL_TYPE, op_ige),
117     Label('seq'): OpDef(Label('seq'), [INT_TYPE, INT_TYPE], BOOL_TYPE, op_seq),
118     Label('sne'): OpDef(Label('sne'), [INT_TYPE, INT_TYPE], BOOL_TYPE, op_sne),
119     Label('slt'): OpDef(Label('slt'), [INT_TYPE, INT_TYPE], BOOL_TYPE, op_slt),
120     Label('sle'): OpDef(Label('sle'), [INT_TYPE, INT_TYPE], BOOL_TYPE, op_sle),
121     Label('sgt'): OpDef(Label('sgt'), [INT_TYPE, INT_TYPE], BOOL_TYPE, op_sgt),
122     Label('sge'): OpDef(Label('sge'), [INT_TYPE, INT_TYPE], BOOL_TYPE, op_sge)}
123 #
124 # end of op.py
125 # Id: op.py 4838 2021-11-14 12:02:46Z
```

A.7 State transition rules

st.py

```
# (c) 2021
# license: GPL-3
#
from bidict import dict_union, bidict_union

from subtype import Value, NULL_TYPE, VALUE_TYPES, subtype

from graph import (
    Mismatch,   Label,      gc_state,   push_state, pop_state,
    add_object_to_state,    add_value_to_state,     swing_state,
    find_var,   find_lvar,  find_attr,  find_lattr, extract_pattern,
    cons_match)

from pattern import PatternConj, PatternDisj, MatchStmt

from asx import (
    AssignStmt, IfStmt,     WhileStmt,  PrintStmt,  BlockStmt,
    VarDecl,    VarEnd,     VarExpr,    AttrExpr,   OpExpr,
    NewExpr,    AndExpr,    OrExpr)

from op import invoke_op

SCOPE_LABEL = Label('$')

def eval_value(x, sg):
    return add_value_to_state(sg, x)

def eval_var(x, sg):
    return find_var(sg, SCOPE_LABEL, x.label)

def eval_lvar(x, sg):
    return find_lvar(sg, SCOPE_LABEL, x.label)

def eval_attr(x, sg):
    p = eval_expr(x.expr, sg)
    return find_attr(sg, p, x.label)

def eval_lattr(x, sg):
    p = eval_expr(x.expr, sg)
    return find_lattr(sg, p, x.label)

def eval_op(x, sg):
    la, args = x
    return invoke_op(sg, la, [eval_expr(arg, sg) for arg in args])

def eval_new(x, sg):
    return add_object_to_state(sg, x.cla)

def eval_and(x, sg):
    left, right = x
    p = eval_expr(left, sg)

    g, tm, vm = sg
    if vm[p].value != True:
        return p

    return eval_expr(right, sg)

def eval_or(x, sg):
    left, right = x
```

```
    p = eval_expr(left, sg)

    g, tm, vm = sg
    if vm[p].value != False:
        return p

    return eval_expr(right, sg)

EXPR_TAB = {
    Value: eval_value,
    VarExpr: eval_var,
    AttrExpr: eval_attr,
    OpExpr: eval_op,
    NewExpr: eval_new,
    AndExpr: eval_and,
    OrExpr: eval_or}

def eval_expr(x, sg):
    return EXPR_TAB[type(x)](x, sg)

LEXPR_TAB = {
    VarExpr: eval_lvar,
    AttrExpr: eval_lattr}

def eval_lexpr(lx, sg):
    return LEXPR_TAB[type(lx)](lx, sg)

def st_var_decl(vd, sg):
    la, cla = vd
    sg = push_state(sg, SCOPE_LABEL)
    q = add_object_to_state(sg, NULL_TYPE)
    swing_state(sg, sg.layout.root, la, q)
    return gc_state(sg)

def st_var_end(ve, sg):
    la = ve.label
    return pop_state(sg, SCOPE_LABEL)

def st_print(pr, sg):
    def to_str(p):
        cla = sg.types[p]

        if cla is NULL_TYPE:
            return 'null'

        if cla in VALUE_TYPES:
            return str(sg.values[p].value)

        return '{}@({})'.format(cla.tag, p.id)

    rs = [to_str(eval_expr(x, sg)) for x in pr.args]
    print(',␣'.join(rs))
    return sg

def st_assign(s, sg):
    lx, x = s
    (p, la) = eval_lexpr(lx, sg)
    q = eval_expr(x, sg)
    swing_state(sg, p, la, q)
    return gc_state(sg)

def st_if(s, sg):
```

```
    x, thens, elses = s
    p = eval_expr(x, sg)
    if sg.values[p].value == True:
        return st_stmt(thens, sg)
    else:
        return st_stmt(elses, sg)

def st_while(s, sg):
    x, ws = s
    p = eval_expr(x, sg)
    while sg.values[p].value == True:
        sg = st_stmt(ws, sg)
        p = eval_expr(x, sg)

    return gc_state(sg)

def st_block(blk, sg):
    for s in blk.stmts:
        sg = st_scope(s, sg)

    return gc_state(sg)

def st_scope(s, sg):
    if type(s) is VarDecl:
        return st_var_decl(s, sg)

    if type(s) is VarEnd:
        return st_var_end(s, sg)

    return st_stmt(s, sg)

def st_match(s, sg):
    x, cas = s
    p = eval_expr(x, sg)
    pg = extract_pattern(sg, p)

    for junc, s, extra in cas:
        m = match_junc(pg, junc, extra.get())
        if m is not None:
            sg = push_state(sg, SCOPE_LABEL)
            for la, q in m:
                swing_state(sg, sg.layout.root, la, q)
            sg = st_stmt(s, sg)
            return pop_state(sg, SCOPE_LABEL)

    return sg

def match_junc(pg, junc, extra):
    if type(junc) is PatternConj:
        pgs1, fs1, rms1 = extra
        return match_conj(pg, pgs1, rms1)

    if type(junc) is PatternDisj:
        pgs1, fs1, rms1 = extra
        return match_disj(pg, pgs1, fs1, rms1)

    pg1, rm1 = extra
    return match_one(pg, pg1, rm1)

def match_patterns(pg1, pg2):
    g1, ts1 = pg1
    g2, ts2 = pg2
```

```
185
186     try:
187         f = cons_match(g2, g1, subtype, ts2, ts1)
188         return f
189     except Mismatch:
190         return None
191
192 def match_one(pg, pg1, rm1):
193     f = match_patterns(pg, pg1)
194
195     if f is None:
196         return None
197
198     return [(la, f[q]) for la, q in rm1.items()]
199
200 def match_disj(pg, pgs1, fs1, rms1):
201     for pg1 in pgs1:
202         f = match_patterns(pg, pg1)
203         if f is not None:
204             rm1 = dict_union(rms1)
205             f1 = bidict_union(fs1)
206             return [(la, f[(set(f1.inv[f1[u]]) & set(f)).pop()]) for la, u in rm1.items()]
207
208     return None
209
210 def match_conj(pg, pgs1, rms1):
211     fs = [match_patterns(pg, pg1) for pg1 in pgs1]
212     if any(f is None for f in fs):
213         return None
214
215     rm1 = dict_union(rms1)
216     f = bidict_union(fs)
217     return [(la, f[u]) for la, u in rm1.items()]
218
219 STMT_TAB = {
220     MatchStmt: st_match,
221     AssignStmt: st_assign,
222     IfStmt: st_if,
223     WhileStmt: st_while,
224     PrintStmt: st_print,
225     BlockStmt: st_block}
226
227 def st_stmt(s, sg):
228     return STMT_TAB[type(s)](s, sg)
229
230 def st_program(prog, sg):
231     return st_block(prog.block, sg)
232 #
233 # end of st.py
234 # Id: st.py 4841 2021-11-14 17:09:10Z
```

A.8 Types and subtyping rules

subtype.py

```
1 # (c) 2021
2 # license: GPL-3
3 #
4 from collections import namedtuple
```

```

from pp import PrfxPP

class SubtypeError(Exception): pass
class DuplicateClassError(SubtypeError): pass
class UndefinedClassError(SubtypeError): pass
class AttrTypeConflictError(SubtypeError): pass
class MinTypeError(SubtypeError): pass

Tag = namedtuple('Tag', ['id'])
Lazy = namedtuple('Lazy', ['tag'])

def is_anon_tag(tag):
    return not tag or tag.id[0] == '*'

CLA_TAB = {}

class Cla:
    def __init__(self, tag, supers = [], attrs = [], tags = []):
        if tag:
            if tag in CLA_TAB:
                raise DuplicateClassError()
            CLA_TAB[tag] = self

        self.tag = tag
        self.tags = set(tags)
        if not is_anon_tag(tag):
            self.tags.add(tag)

        self.attrs = dict(attrs)

        for su in supers:
            self.tags.update(su.tags)
            for la, ty in su.attrs.items():
                if la in self.attrs:
                    if self.attrs[la] != ty:
                        raise AttrTypeConflictError()
                else:
                    self.attrs[la] = ty

        self.hashcode_cache = self.compute_hash()

    def compute_hash(self):
        s = list(self.tags)
        s.sort()
        return hash(('Cla', tuple(s)))

    def resolve_lazy(self):
        lzs = [(la, ty) for la, ty in self.attrs.items() if type(ty) is Lazy]
        for la, ty in lzs:
            self.attrs[la] = CLA_TAB[ty.tag]
        return self

    def __eq__(self, y):
        if self is y:
            return True

        if not is_anon_tag(self.tag) and self.tag == y.tag:
            return True

        return self.tags == y.tags

```

```
67      def __hash__(self):
68          return self.hashcode_cache
69
70      def __le__(self, y):
71          if self is y:
72              return True
73
74          if not is_anon_tag(self.tag) and self.tag == y.tag:
75              return True
76
77          return self.tags >= y.tags
78
79      def to_pp(self):
80          pp = {}
81          pp['class_tag'] = None if self.tag is None else self.tag.id
82          pp['supers'] = sorted(tag.id for tag in self.tags)
83          pp['attrs'] = {la.id: cla.tag.id for la, cla in self.attrs.items()}
84          return PrfxPP('Cla', pp)
85
86      @staticmethod
87      def get(tag):
88          if tag not in CLA_TAB:
89              raise UndefinedClassError()
90          return CLA_TAB[tag]
91
92      @staticmethod
93      def reset():
94          CLA_TAB = {}
95
96      @staticmethod
97      def inter(cs):
98          return Cla(None, cs)
99
100     @staticmethod
101     def union(cs):
102         tags = set.intersection(*(cla.tags for cla in cs))
103         attrs = set.intersection(*(set(cla.attrs.items()) for cla in cs))
104         return Cla(None, [], attrs, tags)
105
106 NO_TYPE = Cla(Tag('*TOP'))
107 NULL_TYPE = Cla(Tag('NULL'), set(), {})
108 INT_TYPE = Cla(Tag('INT'), set(), {})
109 STR_TYPE = Cla(Tag('STR'), set(), {})
110 BOOL_TYPE = Cla(Tag('BOOL'), set(), {})
111
112 VALUE_TYPES = {INT_TYPE, STR_TYPE, BOOL_TYPE}
113
114 Value = namedtuple('Value', ['cla', 'value'])
115
116 class ValueSet:
117     def __init__(self, vs):
118         self.values = set(vs)
119         s = list(self.values)
120         s.sort()
121         self.vector = tuple(s)
122
123     def __hash__(self):
124         return hash(('ValueSet', self.vector))
125
126     def __eq__(self, y):
127         return self.vector == y.vector
128
```

```
129 def subtype(x, y):
130     if x is NULL_TYPE:
131         return True
132 
133     if type(x) is Cla and type(y) is Cla:
134         return x <= y
135 
136     if type(x) is ValueSet and type(y) is ValueSet:
137         return x.values <= y.values
138 
139     if type(x) is ValueSet and type(y) is Cla:
140         return all(v.cla == y for v in x.values)
141 
142     return False
143 
144 def min_type(x, y):
145     if subtype(x, y):
146         return x
147 
148     if subtype(y, x):
149         return y
150 
151     raise MinTypeError()
152 
153 def classof(x, path = []):
154     if type(x) is ValueSet:
155         if path:
156             return None
157         clas = set(v.cla for v in x.values)
158         if len(clas) != 1:
159             return None
160         return clas.pop()
161 
162     if type(x) is Cla:
163         cla = x
164         for na in path:
165             if na not in cla.attrs:
166                 return None
167             cla = cla.attrs[na]
168         return cla
169 
170     return None
171 
172 def exists_ty_le_all(ts):
173     clas = [classof(t) for t in ts]
174     if any(cla is None for cla in clas):
175         return False
176 
177     if any(cla in VALUE_TYPES for cla in clas):
178         return len(set(clas)) == 1
179 
180     return True
181 
182 def ty_inf(ts):
183     clas = [classof(t) for t in ts]
184     if any(cla in VALUE_TYPES for cla in clas):
185         vss = [t for t in ts if type(t) is ValueSet]
186         if vss:
187             return ValueSet(set().union(*[vs.values for vs in vss]))
188         else:
189             return clas[0]
190     else:
```

```
191         return Cla.inter(clas)
192
193 def ty_sup(ts):
194     clas = [classof(t) for t in ts]
195     if any(cla in VALUE_TYPES for cla in clas):
196         vss = [t for t in ts if type(t) is ValueSet]
197         if vss:
198             return ValueSet(set().intersection(*[vs.values for vs in vss]))
199         else:
200             return clas[0]
201     else:
202         return Cla.union(clas)
203 #
204 # end of subtype.py
205 # Id: subtype.py 4838 2021-11-14 12:02:46Z
```

A.9 Type-checking rules

tc.py

```
1  # (c) 2021
2  # license: GPL-3
3  #
4  from subtype import Cla, Value, subtype, min_type, classof, BOOL_TYPE, VALUE_TYPES
5
6  from op import is_op, get_op
7
8  from asx import (
9      PrintStmt,  AssignStmt, IfStmt,     WhileStmt,  VarDecl,
10     VarEnd,     VarExpr,    AttrExpr,   OpExpr,     NewExpr,
11     AndExpr,    OrExpr,     Program)
12
13 from pattern import PatternConj, PatternDisj, MatchStmt
14
15 from graph import (
16     PatternGraph, cons_pattern_graph, cons_match_conj,
17     cons_match_disj, unzip4)
18
19 class Env:
20     def __init__(self, outer = None, items = []):
21         self.tab = dict(items)
22         self.outer = outer
23
24     def __contains__(self, k):
25         return k in self.tab or (self.outer is not None and k in self.outer)
26
27     def __getitem__(self, k):
28         if k in self.tab:
29             return self.tab[k]
30         else:
31             return self.outer[k]
32
33     def __setitem__(self, k, v):
34         self.tab[k] = v
35
36     def top(self):
37         return set(self.tab)
38
39     def pop(self):
```

```
        return self.outer

    def show(self):
        print(self.tab)
        env = self.outer
        while env is not None:
            print(env.tab)
            env = env.outer

class TypeCheckingError(Exception): pass
class NodeTypeError(TypeCheckingError): pass
class NodeSubtypeError(TypeCheckingError): pass
class IncompatibleTypesError(TypeCheckingError): pass
class UndefVarError(TypeCheckingError): pass
class ParentObjectError(TypeCheckingError): pass
class AttrError(TypeCheckingError): pass
class OpError(TypeCheckingError): pass
class OpArgLenError(TypeCheckingError): pass
class OpArgTypeError(TypeCheckingError): pass
class VarEndError(TypeCheckingError): pass
class LeftExprError(TypeCheckingError): pass
class ClassError(TypeCheckingError): pass
class AssignTypeError(TypeCheckingError): pass
class CondTypeError(TypeCheckingError): pass
class ScopeError(TypeCheckingError): pass
class PrintArgTypeError(TypeCheckingError): pass
class BoolTypeError(TypeCheckingError): pass

def tc_node(p, env, es, types):
    if p in env:
        return env[p]

    if p not in types:
        raise NodeTypeError()

    t = types[p]
    env2 = Env(env, {p: t})

    for la in es.labels[p]:
        q = es.targets[(p, la)]
        s = tc_node(q, env2, es, types)
        if not subtype(s, classof(t, [la])):
            raise NodeSubtypeError()

    return t

def tc_graph(pg, env):
    (ns, es, p), types = pg
    t = tc_node(p, env, es, types)
    return t

def tc_ref(la, env, pg, refs):
    p = refs[la]
    (ns, es, p2), types = pg
    t = tc_node(p, env, es, types)
    return t

def tc_pattern(pattern, env):
    g, tm, rm = cons_pattern_graph(pattern)
    pg = PatternGraph(g, tm)
    t = tc_graph(pg, env)
    rtm = {la: tc_ref(la, env, pg, rm) for la in rm}
```

```
102     return (t, rtm, pg, rm)
103
104 def tc_conj(patterns, env):
105     ts, rtms, pgs, rms = unzip4([tc_pattern(pattern, env) for pattern in patterns])
106     fs, tsd = cons_match_conj(pgs)
107     rtm2 = {la: min_type(s, tsd[f[rm[la]]])
108             for f, rtm, rm in zip(fs, rtms, rms) for la, s in rtm.items()}
109     th = tsd[fs[0][pgs[0].layout.root]]
110     pts = [min_type(t, th) for t in ts]
111     return (th, pts, rtm2, pgs, fs, rms)
112
113 def tc_disj(patterns, env):
114     ts, rtms, pgs, rms = unzip4([tc_pattern(pattern, env) for pattern in patterns])
115     fs, tsc = cons_match_disj(pgs)
116     rtm2 = {la: tsc[f[rm[la]]] for f, rtm, rm in zip(fs, rtms, rms) for la in rtm}
117     th = tsc[fs[0][pgs[0].layout.root]]
118     pts = [t for t in ts]
119     return (th, pts, rtm2, pgs, fs, rms)
120
121 def tc_case(ca, env):
122     junc, stmt, extra = ca
123
124     if type(junc) is PatternConj:
125         t, pts, rtm, pgs, fs, rms = tc_conj(junc.patterns, env)
126         extra.put((pgs, fs, rms))
127     elif type(junc) is PatternDisj:
128         t, pts, rtm, pgs, fs, rms = tc_disj(junc.patterns, env)
129         extra.put((pgs, fs, rms))
130     else:
131         t, rtm, pg, rm = tc_pattern(junc, env)
132         extra.put((pg, rm))
133
134     env2 = Env(env, rtm)
135
136     tc_stmt(stmt, env2)
137     return t
138
139 def tc_value(v, env):
140     return v.cla
141
142 def tc_var(x, env):
143     la = x.label
144
145     if la not in env:
146         print(la)
147         env.show()
148         raise UndefVarError()
149
150     return env[la]
151
152 def tc_attr(x, env):
153     y, la = x
154     t = tc_expr(y, env)
155
156     if type(t) is not Cla:
157         raise ParentObjectError()
158
159     if la not in t.attrs:
160         print(la)
161         raise AttrError()
162
163     return t.attrs[la]
```

```
164
165 def tc_op(x, env):
166     op, args = x
167     if not is_op(op):
168         raise OpError()
169
170     d = get_op(op)
171     if len(args) != len(d.par_types):
172         raise OpArgLenError()
173
174     for y, pty in zip(args, d.par_types):
175         if tc_expr(y, env) != pty:
176             raise OpArgTypeError()
177
178     return d.res_type
179
180 def tc_new(x, env):
181     if type(x.cla) is not Cla:
182         raise ClassError()
183
184     return x.cla
185
186 def tc_and_or(x, env):
187     left, right = x
188
189     if tc_expr(left, env) is not BOOL_TYPE or tc_expr(right, env) is not BOOL_TYPE:
190         raise BoolTypeError()
191
192     return BOOL_TYPE
193
194 EXPR_TAB = {
195     Value: tc_value,
196     VarExpr: tc_var,
197     AttrExpr: tc_attr,
198     OpExpr: tc_op,
199     NewExpr: tc_new,
200     AndExpr: tc_and_or,
201     OrExpr: tc_and_or}
202
203 def tc_expr(x, env):
204     return EXPR_TAB[type(x)](x, env)
205
206 def tc_match(m, env):
207     expr, cas = m
208     t2 = tc_expr(expr, env)
209
210     for ca in cas:
211         t = tc_case(ca, env)
212         if not (subtype(t2, t) or subtype(t, t2)):
213             raise IncompatibleTypesError()
214
215     return env
216
217 def tc_var_decl(vd, env):
218     la, cla = vd
219     return Env(env, {la: cla})
220
221 def tc_var_end(ve, env, env_base):
222     if env is env_base:
223         raise VarEndError()
224
225     if set([ve.label]) != env.top():
```

```
226             raise VarEndError()
227
228         return env.pop()
229
230     def tc_print(pr, env):
231         for x in pr.args:
232             tc_expr(x, env)
233
234         return env
235
236     def tc_assign(assign, env):
237         lx, x = assign
238
239         if type(lx) not in {VarExpr, AttrExpr}:
240             raise LeftExprError()
241
242         t = tc_expr(lx, env)
243         t2 = tc_expr(x, env)
244
245         if not subtype(t2, t):
246             raise AssignTypeError()
247
248         return env
249
250     def tc_if(ifs, env):
251         x, thens, elses = ifs
252
253         if tc_expr(x, env) is not BOOL_TYPE:
254             raise CondTypeError()
255
256         tc_stmt(thens, env)
257         tc_stmt(elses, env)
258         return env
259
260     def tc_while(ws, env):
261         x, s = ws
262
263         if tc_expr(x, env) is not BOOL_TYPE:
264             raise CondTypeError()
265
266         tc_stmt(s, env)
267         return env
268
269     def tc_stmt(s, env):
270         if type(s) is MatchStmt:
271             return tc_match(s, env)
272
273         if type(s) is AssignStmt:
274             return tc_assign(s, env)
275
276         if type(s) is IfStmt:
277             return tc_if(s, env)
278
279         if type(s) is WhileStmt:
280             return tc_while(s, env)
281
282         if type(s) is PrintStmt:
283             return tc_print(s, env)
284
285         return tc_block(s, env)
286
287     def tc_block(blk, env):
```

```
288     env2 = env
289     for s in blk.stmts:
290         env2 = tc_scope(s, env2, env)
291 
292     if env2 is not env:
293         raise ScopeError()
294 
295     return env
296 
297 def tc_scope(s, env, env_base):
298     if type(s) is VarDecl:
299         return tc_var_decl(s, env)
300 
301     if type(s) is VarEnd:
302         return tc_var_end(s, env, env_base)
303 
304     return tc_stmt(s, env)
305 
306 def tc_program(prog, env):
307     return tc_block(prog.block, env)
308 #
309 # end of tc.py
310 # Id: tc.py 4841 2021-11-14 17:09:10Z
```

A.10 Test cases

test_cases.py

```
1  # (c) 2021
2  # license: GPL-3
3  #
4  from pp import pprint
5 
6  from subtype import (
7      Tag,        Lazy,       Cla,        subtype,    INT_TYPE,
8      STR_TYPE,   BOOL_TYPE,  NO_TYPE,    Value,      ValueSet)
9 
10 from pattern import (
11     LabeledPattern, ClassPattern,   PatternRef, PatternConj,
12     PatternDisj,    MatchStmt,      Case,       Extra)
13 
14 from graph import (
15     Label,  cons_pattern_graph, layout_graph_to_pp,
16     init_state_graph)
17 
18 from asx import (
19     VarDecl,    VarEnd,     PrintStmt,  AssignStmt, BlockStmt,
20     IfStmt,     WhileStmt,  VarExpr,    AttrExpr,   NewExpr,
21     OpExpr,     AndExpr,    OrExpr,     Program)
22 
23 from tc import tc_stmt, tc_program, Env
24 
25 from st import st_stmt, st_program
26 
27 def test_subtype():
28     print(
29 '''
30 ----
31 ----␣subtype␣----
```

```
----
''')

    Cla.reset()

    a1 = Label('a1')
    a2 = Label('a2')
    b1 = Label('b1')
    b2 = Label('b2')
    c1 = Label('c1')
    c2 = Label('c2')
    d1 = Label('d1')
    d2 = Label('d2')

    A = Cla(Tag('A'),
        [],
        {
            a1: INT_TYPE,
            a2: STR_TYPE
        })

    B = Cla(Tag('B'),
        [A],
        {
            b1: INT_TYPE,
            b2: STR_TYPE
        })

    C = Cla(Tag('C'),
        [A],
        {
            c1: INT_TYPE,
            c2: STR_TYPE
        })

    D = Cla(Tag('D'),
        [B, C],
        {
            d1: INT_TYPE,
            d2: STR_TYPE
        })

    pprint(D.to_pp())

    E = Cla.inter([B, C])
    pprint(E.to_pp())

    print(subtype(D, E))
    print(subtype(E, D))

    pprint(NO_TYPE.to_pp())
    print(subtype(A, NO_TYPE))

    ONE = Value(INT_TYPE, 1)
    TWO = Value(INT_TYPE, 2)
    THREE = Value(INT_TYPE, 3)
    TEN = Value(INT_TYPE, 10)
    HELLO = Value(STR_TYPE, 'Hello')

    VSET1 = ValueSet(set(Value(INT_TYPE, x) for x in range(10)))
    print(subtype(ValueSet(set([ONE, TWO])), VSET1))
    print(subtype(ValueSet(set([ONE, TWO, TEN])), VSET1))
```

```
94      print(subtype(VSET1, INT_TYPE))
95
96      F = Cla.union([B, C])
97      pprint(F.to_pp())
98      print(subtype(F, A))
99      print(subtype(A, F))
100     print(A == F)
101
102     h1 = Label('h1')
103     h2 = Label('h2')
104     k = Label('k')
105
106     H = Cla(Tag('H'),
107         [],
108         {
109             h1: INT_TYPE,
110             h2: A
111         })
112
113     K = Cla(Tag('K'),
114         [],
115         {
116             k: H
117         })
118
119     o = Label('o')
120     q = Label('q')
121     x = Label('x')
122     y = Label('y')
123
124     pb = ClassPattern(
125         K,
126         {
127             k : ClassPattern(
128                 H,
129                 {
130                     h1: ClassPattern(INT_TYPE, {}),
131                     h2: ClassPattern(B, {})
132                 })
133         })
134
135     pc = ClassPattern(
136         K,
137         {
138             k : ClassPattern(
139                 H,
140                 {
141                     h1: ClassPattern(INT_TYPE, {}),
142                     h2: LabeledPattern(x, ClassPattern(C, {}))
143                 })
144         })
145
146     s1 = BlockStmt([
147         VarDecl(o, K),
148         VarDecl(q, D),
149         AssignStmt(VarExpr(o), NewExpr(K)),
150         AssignStmt(AttrExpr(VarExpr(o), k), NewExpr(H)),
151         AssignStmt(AttrExpr(AttrExpr(VarExpr(o), k), h1), Value(INT_TYPE, 3)),
152         AssignStmt(VarExpr(q), NewExpr(D)),
153         AssignStmt(AttrExpr(VarExpr(q), a1), Value(INT_TYPE, 4)),
154         AssignStmt(AttrExpr(VarExpr(q), b1), Value(INT_TYPE, 5)),
155         AssignStmt(AttrExpr(VarExpr(q), c1), Value(INT_TYPE, 6)),
```

```
156         AssignStmt(AttrExpr(AttrExpr(VarExpr(o), k), h2), VarExpr(q)),
157
158         MatchStmt(VarExpr(o), [
159             Case(PatternConj([pb, pc]), BlockStmt([
160                 VarDecl(y, E),
161                 AssignStmt(VarExpr(y), VarExpr(x)),
162                 PrintStmt([
163                     AttrExpr(VarExpr(y), a1),
164                     AttrExpr(VarExpr(y), b1),
165                     AttrExpr(VarExpr(y), c1)
166                     ]),
167                 VarEnd(y)
168                 ]), Extra())
169             ]),
170
171         MatchStmt(VarExpr(o), [
172             Case(PatternDisj([pb, pc]), BlockStmt([
173                 VarDecl(y, F),
174                 AssignStmt(VarExpr(y), VarExpr(x)),
175                 PrintStmt([AttrExpr(VarExpr(y), a1)]),
176                 VarEnd(y)
177                 ]), Extra())
178             ]),
179
180         VarEnd(q),
181         VarEnd(o)])
182
183     env = tc_stmt(s1, Env())
184     sg = st_stmt(s1, init_state_graph())
185
186
187 def test_fig2():
188     print(
189 '''
190 ----
191 ----␣figure␣2␣----
192 ----
193 ''')
194
195     Cla.reset()
196
197     e = Label('e')
198     l = Label('l')
199     r = Label('r')
200
201     w = Label('w')
202     x = Label('x')
203     y = Label('y')
204     z = Label('z')
205
206     T_lz = Lazy(Tag('T'))
207
208     T = Cla(T_lz.tag,
209         [],
210         {
211             e: INT_TYPE,
212             l: T_lz,
213             r: T_lz
214         }).resolve_lazy()
215
216     pprint(T.to_pp())
217
```

```
    pattern = ClassPattern(
        T,
        {
            e: ValueSet({Value(INT_TYPE, 0)}),
            l: LabeledPattern(w, ClassPattern(
                T,
                {
                    l: LabeledPattern(x, ClassPattern(T, {})),
                    r: LabeledPattern(y, ClassPattern(
                        T,
                        {
                            l: PatternRef(w),
                            r: PatternRef(z)
                        }))
                })),
            r: LabeledPattern(z, ClassPattern(T, {}))
        })

    g, tm, rm = cons_pattern_graph(pattern)
    pprint(layout_graph_to_pp(g))

    o = Label('o')

    s1 = BlockStmt([
        VarDecl(o, T),
        AssignStmt(VarExpr(o), NewExpr(T)),
        AssignStmt(AttrExpr(VarExpr(o), l), NewExpr(T)),
        AssignStmt(AttrExpr(VarExpr(o), r), NewExpr(T)),
        AssignStmt(AttrExpr(AttrExpr(VarExpr(o), l), l), NewExpr(T)),
        AssignStmt(AttrExpr(AttrExpr(VarExpr(o), l), r), NewExpr(T)),
        AssignStmt(AttrExpr(AttrExpr(AttrExpr(VarExpr(o), l), r), l), AttrExpr(VarExpr(o), l)),
        AssignStmt(AttrExpr(AttrExpr(AttrExpr(VarExpr(o), l), r), r), AttrExpr(VarExpr(o), r)),

        AssignStmt(AttrExpr(VarExpr(o), e), Value(INT_TYPE, 0)),
        AssignStmt(AttrExpr(AttrExpr(VarExpr(o), l), e), Value(INT_TYPE, 1)),
        AssignStmt(AttrExpr(AttrExpr(AttrExpr(VarExpr(o), l), l), e), Value(INT_TYPE, 2)),
        AssignStmt(AttrExpr(AttrExpr(AttrExpr(VarExpr(o), l), r), e), Value(INT_TYPE, 3)),
        AssignStmt(AttrExpr(AttrExpr(VarExpr(o), r), e), Value(INT_TYPE, 4)),

        MatchStmt(VarExpr(o), [
            Case(pattern, PrintStmt([
                AttrExpr(VarExpr(w), e),
                AttrExpr(VarExpr(x), e),
                AttrExpr(VarExpr(y), e),
                AttrExpr(VarExpr(z), e)
                ]), Extra())
            ]),
        VarEnd(o)])

    env = tc_stmt(s1, Env())
    sg = st_stmt(s1, init_state_graph())


def test_fig3():
    print(
'''
----
----␣figure␣3␣----
----
''')

    Cla.reset()
```

```
280
281     a = Label('a')
282     b = Label('b')
283     c = Label('c')
284     d = Label('d')
285     e = Label('e')
286     x = Label('x')
287     y = Label('y')
288
289     X = Cla(Tag('X'),
290         [],
291         {
292             b: INT_TYPE,
293             c: STR_TYPE,
294             d: STR_TYPE
295         })
296
297     pprint(X.to_pp())
298
299     Y = Cla(Tag('Y'),
300         [X],
301         {
302             a: INT_TYPE,
303         })
304
305     pprint(Y.to_pp())
306
307     Z = Cla(Tag('Z'),
308         [X],
309         {
310             e: BOOL_TYPE,
311         })
312
313     pprint(Z.to_pp())
314
315     W = Cla(Tag('W'),
316         [Y, Z],
317         {})
318
319     pprint(W.to_pp())
320
321     p1 = ClassPattern(
322         Y,
323         {
324             a: ValueSet(set(Value(INT_TYPE, x) for x in range(3))),
325             c: LabeledPattern(x, ClassPattern(STR_TYPE, {})),
326             d: ClassPattern(STR_TYPE, {})
327         })
328
329     p2 = ClassPattern(
330         Z,
331         {
332             c: ClassPattern(STR_TYPE, {}),
333             d: ClassPattern(STR_TYPE, {}),
334             e: ValueSet({Value(BOOL_TYPE, False)})
335         })
336
337     p3 = ClassPattern(
338         X,
339         {
340             b: ValueSet({Value(INT_TYPE, 1)}),
341             c: ValueSet({Value(STR_TYPE, 'apple')}),
```

```
            d: ValueSet({Value(STR_TYPE, 'banana')})
        })

    p4 = ClassPattern(
        Z,
        {
            b: ValueSet({Value(INT_TYPE, 1)}),
            c: LabeledPattern(y, ValueSet({Value(STR_TYPE, 'apple')})),
            d: PatternRef(y)
        })

    conj = PatternConj([p1, p2, p3])
    disj = PatternDisj([p3, p4])

    o = Label('o')
    q = Label('q')
    u = Label('u')
    v = Label('v')

    prog = Program(BlockStmt([
        VarDecl(o, W),
        VarDecl(q, STR_TYPE),
        VarDecl(u, BOOL_TYPE),
        VarDecl(v, INT_TYPE),
        AssignStmt(VarExpr(o), NewExpr(W)),
        AssignStmt(AttrExpr(VarExpr(o), a), Value(INT_TYPE, 1)),
        AssignStmt(AttrExpr(VarExpr(o), b), AttrExpr(VarExpr(o), a)),
        AssignStmt(AttrExpr(VarExpr(o), c), Value(STR_TYPE, 'apple')),
        AssignStmt(AttrExpr(VarExpr(o), d), Value(STR_TYPE, 'banana')),
        AssignStmt(AttrExpr(VarExpr(o), e), Value(BOOL_TYPE, False)),

        MatchStmt(VarExpr(o), [
            Case(p4, PrintStmt([Value(STR_TYPE, 'p4, This should not match.')]), Extra())
            ]),
        PrintStmt([Value(STR_TYPE, 'after p4'), VarExpr(q)]),
        MatchStmt(VarExpr(o), [
            Case(p4, PrintStmt([Value(STR_TYPE, 'p4, This should not match.')]), Extra()),
            Case(disj, BlockStmt([
                AssignStmt(VarExpr(q), VarExpr(y)),
                PrintStmt([Value(STR_TYPE, 'disj'), VarExpr(q)])]), Extra())
            ]),
        MatchStmt(VarExpr(o), [
            Case(conj, BlockStmt([
                AssignStmt(VarExpr(q), VarExpr(x)),
                PrintStmt([Value(STR_TYPE, 'conj'), VarExpr(q)])]), Extra()),
            Case(p1, BlockStmt([
                AssignStmt(VarExpr(q), VarExpr(x)),
                PrintStmt([Value(STR_TYPE, 'p1'), VarExpr(q)])]), Extra())
            ]),
        MatchStmt(VarExpr(o), [
            Case(p4, PrintStmt([Value(STR_TYPE, 'p4, This should not match.')]), Extra()),
            Case(p1, BlockStmt([
                AssignStmt(VarExpr(q), VarExpr(x)),
                PrintStmt([Value(STR_TYPE, 'p1'), VarExpr(q)])]), Extra())
            ]),
        MatchStmt(VarExpr(o), [
            Case(p4, PrintStmt([Value(STR_TYPE, 'p4, This should not match.')]), Extra()),
            Case(p2, BlockStmt([
                AssignStmt(VarExpr(v), AttrExpr(VarExpr(o), b)),
                PrintStmt([Value(STR_TYPE, 'p2'), VarExpr(v)])]), Extra())
            ]),
        MatchStmt(VarExpr(o), [
```

```
404             Case(p4, PrintStmt([Value(STR_TYPE, 'p4, This should not match.')]), Extra()),
405             Case(p3, BlockStmt([
406                 AssignStmt(VarExpr(u), AttrExpr(VarExpr(o), e)),
407                 PrintStmt([Value(STR_TYPE, 'p3'), VarExpr(u)])]), Extra())
408             ]),
409         VarEnd(v),
410         VarEnd(u),
411         VarEnd(q),
412         VarEnd(o)]))
413
414     env = tc_program(prog, Env())
415     sg = st_program(prog, init_state_graph())
416
417
418 def test_gcd():
419     print(
420 '''
421 ----
422 ---- gcd ----
423 ----
424 ''')
425
426     Cla.reset()
427
428     m = Label('m')
429     n = Label('n')
430     t = Label('t')
431
432     s1 = BlockStmt([
433         VarDecl(m, INT_TYPE),
434         VarDecl(n, INT_TYPE),
435         AssignStmt(VarExpr(m), Value(INT_TYPE, 210)),
436         AssignStmt(VarExpr(n), Value(INT_TYPE, 120)),
437         PrintStmt([Value(STR_TYPE, 'numbers'), VarExpr(m), VarExpr(n)]),
438
439         IfStmt(
440             OrExpr(
441                 OpExpr(Label('igt'), [VarExpr(m), Value(INT_TYPE, 200)]),
442                 OpExpr(Label('igt'), [VarExpr(n), Value(INT_TYPE, 200)])),
443             PrintStmt([Value(STR_TYPE, 'one is > 200')]),
444             PrintStmt([Value(STR_TYPE, 'none is > 200')])),
445
446         IfStmt(
447             AndExpr(
448                 OpExpr(Label('igt'), [VarExpr(m), Value(INT_TYPE, 200)]),
449                 OpExpr(Label('igt'), [VarExpr(n), Value(INT_TYPE, 200)])),
450             PrintStmt([Value(STR_TYPE, 'both are > 200')]),
451             PrintStmt([Value(STR_TYPE, 'one is <= 200')])),
452
453         BlockStmt([
454             WhileStmt(
455                 OpExpr(Label('ine'), [VarExpr(n), Value(INT_TYPE, 0)]),
456                 BlockStmt([
457                     VarDecl(t, INT_TYPE),
458                     AssignStmt(VarExpr(t), VarExpr(m)),
459                     AssignStmt(VarExpr(m), VarExpr(n)),
460                     AssignStmt(VarExpr(n), OpExpr(Label('mod'), [VarExpr(t), VarExpr(n)])),
461                     VarEnd(t)])),
462             PrintStmt([Value(STR_TYPE, 'gcd'), VarExpr(m)])
463         ]),
464
465         VarEnd(n),
```

```
466         VarEnd(m)
467         ])
468
469     env = tc_stmt(s1, Env())
470     sg = st_stmt(s1, init_state_graph())
471
472
473 if __name__ == '__main__':
474     test_subtype()
475     test_fig2()
476     test_fig3()
477     test_gcd()
478 #
479 # end of test_cases.py
480 # Id: test_cases.py 4841 2021-11-14 17:09:10Z
```

Appendix B.

GNU GENERAL PUBLIC LICENSE TEXT

GNU GENERAL PUBLIC LICENSE

Version 3, 29 June 2007

Copyright © 2007 Free Software Foundation, Inc. http://fsf.org/

Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

Preamble

The GNU General Public License is a free, copyleft license for software and other kinds of works.

The licenses for most software and other practical works are designed to take away your freedom to share and change the works. By contrast, the GNU General Public License is intended to guarantee your freedom to share and change all versions of a program–to make sure it remains free software for all its users. We, the Free Software Foundation, use the GNU General Public License for most of our software; it applies also to any other work released this way by its authors. You can apply it to your programs, too.

When we speak of free software, we are referring to freedom, not price. Our General Public Licenses are designed to make sure that you have the freedom to distribute copies of free software (and charge for them if you wish), that you receive source code or can get it if you want it, that you can change the software or use pieces of it in new free programs, and that you know you can do these things.

To protect your rights, we need to prevent others from denying you these rights or asking you to surrender the rights. Therefore, you have certain responsibilities if you distribute copies of the software, or if you modify it: responsibilities to respect the freedom of others.

For example, if you distribute copies of such a program, whether gratis or for a fee, you must pass on to the recipients the same freedoms that you received. You must make sure that they, too, receive or can get the source code. And you must show them these terms so they know their rights.

Developers that use the GNU GPL protect your rights with two steps: (1) assert copyright on the software, and (2) offer you this License giving you legal permission to copy, distribute and/or modify it.

For the developers' and authors' protection, the GPL clearly explains that there is no warranty for this free software. For both users' and authors' sake, the GPL requires that

modified versions be marked as changed, so that their problems will not be attributed erroneously to authors of previous versions.

Some devices are designed to deny users access to install or run modified versions of the software inside them, although the manufacturer can do so. This is fundamentally incompatible with the aim of protecting users' freedom to change the software. The systematic pattern of such abuse occurs in the area of products for individuals to use, which is precisely where it is most unacceptable. Therefore, we have designed this version of the GPL to prohibit the practice for those products. If such problems arise substantially in other domains, we stand ready to extend this provision to those domains in future versions of the GPL, as needed to protect the freedom of users.

Finally, every program is threatened constantly by software patents. States should not allow patents to restrict development and use of software on general-purpose computers, but in those that do, we wish to avoid the special danger that patents applied to a free program could make it effectively proprietary. To prevent this, the GPL assures that patents cannot be used to render the program non-free.

The precise terms and conditions for copying, distribution and modification follow.

Terms and Conditions

0. Definitions.

   "This License" refers to version 3 of the GNU General Public License.

   "Copyright" also means copyright-like laws that apply to other kinds of works, such as semiconductor masks.

   "The Program" refers to any copyrightable work licensed under this License. Each licensee is addressed as "you". "Licensees" and "recipients" may be individuals or organizations.

   To "modify" a work means to copy from or adapt all or part of the work in a fashion requiring copyright permission, other than the making of an exact copy. The resulting work is called a "modified version" of the earlier work or a work "based on" the earlier work.

   A "covered work" means either the unmodified Program or a work based on the Program.

   To "propagate" a work means to do anything with it that, without permission, would make you directly or secondarily liable for infringement under applicable copyright law, except executing it on a computer or modifying a private copy. Propagation includes copying, distribution (with or without modification), making available to the public, and in some countries other activities as well.

To "convey" a work means any kind of propagation that enables other parties to make or receive copies. Mere interaction with a user through a computer network, with no transfer of a copy, is not conveying.

An interactive user interface displays "Appropriate Legal Notices" to the extent that it includes a convenient and prominently visible feature that (1) displays an appropriate copyright notice, and (2) tells the user that there is no warranty for the work (except to the extent that warranties are provided), that licensees may convey the work under this License, and how to view a copy of this License. If the interface presents a list of user commands or options, such as a menu, a prominent item in the list meets this criterion.

1. Source Code.

   The "source code" for a work means the preferred form of the work for making modifications to it. "Object code" means any non-source form of a work.

   A "Standard Interface" means an interface that either is an official standard defined by a recognized standards body, or, in the case of interfaces specified for a particular programming language, one that is widely used among developers working in that language.

   The "System Libraries" of an executable work include anything, other than the work as a whole, that (a) is included in the normal form of packaging a Major Component, but which is not part of that Major Component, and (b) serves only to enable use of the work with that Major Component, or to implement a Standard Interface for which an implementation is available to the public in source code form. A "Major Component", in this context, means a major essential component (kernel, window system, and so on) of the specific operating system (if any) on which the executable work runs, or a compiler used to produce the work, or an object code interpreter used to run it.

   The "Corresponding Source" for a work in object code form means all the source code needed to generate, install, and (for an executable work) run the object code and to modify the work, including scripts to control those activities. However, it does not include the work's System Libraries, or general-purpose tools or generally available free programs which are used unmodified in performing those activities but which are not part of the work. For example, Corresponding Source includes interface definition files associated with source files for the work, and the source code for shared libraries and dynamically linked subprograms that the work is specifically designed to require, such as by intimate data communication or control flow between those subprograms and other parts of the work.

   The Corresponding Source need not include anything that users can regenerate

automatically from other parts of the Corresponding Source.

The Corresponding Source for a work in source code form is that same work.

2. Basic Permissions.

   All rights granted under this License are granted for the term of copyright on the Program, and are irrevocable provided the stated conditions are met. This License explicitly affirms your unlimited permission to run the unmodified Program. The output from running a covered work is covered by this License only if the output, given its content, constitutes a covered work. This License acknowledges your rights of fair use or other equivalent, as provided by copyright law.

   You may make, run and propagate covered works that you do not convey, without conditions so long as your license otherwise remains in force. You may convey covered works to others for the sole purpose of having them make modifications exclusively for you, or provide you with facilities for running those works, provided that you comply with the terms of this License in conveying all material for which you do not control copyright. Those thus making or running the covered works for you must do so exclusively on your behalf, under your direction and control, on terms that prohibit them from making any copies of your copyrighted material outside their relationship with you.

   Conveying under any other circumstances is permitted solely under the conditions stated below. Sublicensing is not allowed; section 10 makes it unnecessary.

3. Protecting Users' Legal Rights From Anti-Circumvention Law.

   No covered work shall be deemed part of an effective technological measure under any applicable law fulfilling obligations under article 11 of the WIPO copyright treaty adopted on 20 December 1996, or similar laws prohibiting or restricting circumvention of such measures.

   When you convey a covered work, you waive any legal power to forbid circumvention of technological measures to the extent such circumvention is effected by exercising rights under this License with respect to the covered work, and you disclaim any intention to limit operation or modification of the work as a means of enforcing, against the work's users, your or third parties' legal rights to forbid circumvention of technological measures.

4. Conveying Verbatim Copies.

   You may convey verbatim copies of the Program's source code as you receive it, in any medium, provided that you conspicuously and appropriately publish on each copy an appropriate copyright notice; keep intact all notices stating that this License and any non-permissive terms added in accord with section 7 apply to the

code; keep intact all notices of the absence of any warranty; and give all recipients a copy of this License along with the Program.

You may charge any price or no price for each copy that you convey, and you may offer support or warranty protection for a fee.

5. Conveying Modified Source Versions.

   You may convey a work based on the Program, or the modifications to produce it from the Program, in the form of source code under the terms of section 4, provided that you also meet all of these conditions:

   (a) The work must carry prominent notices stating that you modified it, and giving a relevant date.

   (b) The work must carry prominent notices stating that it is released under this License and any conditions added under section 7. This requirement modifies the requirement in section 4 to "keep intact all notices".

   (c) You must license the entire work, as a whole, under this License to anyone who comes into possession of a copy. This License will therefore apply, along with any applicable section 7 additional terms, to the whole of the work, and all its parts, regardless of how they are packaged. This License gives no permission to license the work in any other way, but it does not invalidate such permission if you have separately received it.

   (d) If the work has interactive user interfaces, each must display Appropriate Legal Notices; however, if the Program has interactive interfaces that do not display Appropriate Legal Notices, your work need not make them do so.

   A compilation of a covered work with other separate and independent works, which are not by their nature extensions of the covered work, and which are not combined with it such as to form a larger program, in or on a volume of a storage or distribution medium, is called an "aggregate" if the compilation and its resulting copyright are not used to limit the access or legal rights of the compilation's users beyond what the individual works permit. Inclusion of a covered work in an aggregate does not cause this License to apply to the other parts of the aggregate.

6. Conveying Non-Source Forms.

   You may convey a covered work in object code form under the terms of sections 4 and 5, provided that you also convey the machine-readable Corresponding Source under the terms of this License, in one of these ways:

   (a) Convey the object code in, or embodied in, a physical product (including a physical distribution medium), accompanied by the Corresponding Source

fixed on a durable physical medium customarily used for software interchange.

(b) Convey the object code in, or embodied in, a physical product (including a physical distribution medium), accompanied by a written offer, valid for at least three years and valid for as long as you offer spare parts or customer support for that product model, to give anyone who possesses the object code either (1) a copy of the Corresponding Source for all the software in the product that is covered by this License, on a durable physical medium customarily used for software interchange, for a price no more than your reasonable cost of physically performing this conveying of source, or (2) access to copy the Corresponding Source from a network server at no charge.

(c) Convey individual copies of the object code with a copy of the written offer to provide the Corresponding Source. This alternative is allowed only occasionally and noncommercially, and only if you received the object code with such an offer, in accord with subsection 6b.

(d) Convey the object code by offering access from a designated place (gratis or for a charge), and offer equivalent access to the Corresponding Source in the same way through the same place at no further charge. You need not require recipients to copy the Corresponding Source along with the object code. If the place to copy the object code is a network server, the Corresponding Source may be on a different server (operated by you or a third party) that supports equivalent copying facilities, provided you maintain clear directions next to the object code saying where to find the Corresponding Source. Regardless of what server hosts the Corresponding Source, you remain obligated to ensure that it is available for as long as needed to satisfy these requirements.

(e) Convey the object code using peer-to-peer transmission, provided you inform other peers where the object code and Corresponding Source of the work are being offered to the general public at no charge under subsection 6d.

A separable portion of the object code, whose source code is excluded from the Corresponding Source as a System Library, need not be included in conveying the object code work.

A “User Product” is either (1) a “consumer product”, which means any tangible personal property which is normally used for personal, family, or household purposes, or (2) anything designed or sold for incorporation into a dwelling. In determining whether a product is a consumer product, doubtful cases shall be resolved

in favor of coverage. For a particular product received by a particular user, "normally used" refers to a typical or common use of that class of product, regardless of the status of the particular user or of the way in which the particular user actually uses, or expects or is expected to use, the product. A product is a consumer product regardless of whether the product has substantial commercial, industrial or non-consumer uses, unless such uses represent the only significant mode of use of the product.

"Installation Information" for a User Product means any methods, procedures, authorization keys, or other information required to install and execute modified versions of a covered work in that User Product from a modified version of its Corresponding Source. The information must suffice to ensure that the continued functioning of the modified object code is in no case prevented or interfered with solely because modification has been made.

If you convey an object code work under this section in, or with, or specifically for use in, a User Product, and the conveying occurs as part of a transaction in which the right of possession and use of the User Product is transferred to the recipient in perpetuity or for a fixed term (regardless of how the transaction is characterized), the Corresponding Source conveyed under this section must be accompanied by the Installation Information. But this requirement does not apply if neither you nor any third party retains the ability to install modified object code on the User Product (for example, the work has been installed in ROM).

The requirement to provide Installation Information does not include a requirement to continue to provide support service, warranty, or updates for a work that has been modified or installed by the recipient, or for the User Product in which it has been modified or installed. Access to a network may be denied when the modification itself materially and adversely affects the operation of the network or violates the rules and protocols for communication across the network.

Corresponding Source conveyed, and Installation Information provided, in accord with this section must be in a format that is publicly documented (and with an implementation available to the public in source code form), and must require no special password or key for unpacking, reading or copying.

7. Additional Terms.

"Additional permissions" are terms that supplement the terms of this License by making exceptions from one or more of its conditions. Additional permissions that are applicable to the entire Program shall be treated as though they were included in this License, to the extent that they are valid under applicable law. If additional permissions apply only to part of the Program, that part may be used separately

under those permissions, but the entire Program remains governed by this License without regard to the additional permissions.

When you convey a copy of a covered work, you may at your option remove any additional permissions from that copy, or from any part of it. (Additional permissions may be written to require their own removal in certain cases when you modify the work.) You may place additional permissions on material, added by you to a covered work, for which you have or can give appropriate copyright permission.

Notwithstanding any other provision of this License, for material you add to a covered work, you may (if authorized by the copyright holders of that material) supplement the terms of this License with terms:

(a) Disclaiming warranty or limiting liability differently from the terms of sections 15 and 16 of this License; or

(b) Requiring preservation of specified reasonable legal notices or author attributions in that material or in the Appropriate Legal Notices displayed by works containing it; or

(c) Prohibiting misrepresentation of the origin of that material, or requiring that modified versions of such material be marked in reasonable ways as different from the original version; or

(d) Limiting the use for publicity purposes of names of licensors or authors of the material; or

(e) Declining to grant rights under trademark law for use of some trade names, trademarks, or service marks; or

(f) Requiring indemnification of licensors and authors of that material by anyone who conveys the material (or modified versions of it) with contractual assumptions of liability to the recipient, for any liability that these contractual assumptions directly impose on those licensors and authors.

All other non-permissive additional terms are considered "further restrictions" within the meaning of section 10. If the Program as you received it, or any part of it, contains a notice stating that it is governed by this License along with a term that is a further restriction, you may remove that term. If a license document contains a further restriction but permits relicensing or conveying under this License, you may add to a covered work material governed by the terms of that license document, provided that the further restriction does not survive such relicensing or conveying.

If you add terms to a covered work in accord with this section, you must place, in

the relevant source files, a statement of the additional terms that apply to those files, or a notice indicating where to find the applicable terms.

Additional terms, permissive or non-permissive, may be stated in the form of a separately written license, or stated as exceptions; the above requirements apply either way.

8. Termination.

   You may not propagate or modify a covered work except as expressly provided under this License. Any attempt otherwise to propagate or modify it is void, and will automatically terminate your rights under this License (including any patent licenses granted under the third paragraph of section 11).

   However, if you cease all violation of this License, then your license from a particular copyright holder is reinstated (a) provisionally, unless and until the copyright holder explicitly and finally terminates your license, and (b) permanently, if the copyright holder fails to notify you of the violation by some reasonable means prior to 60 days after the cessation.

   Moreover, your license from a particular copyright holder is reinstated permanently if the copyright holder notifies you of the violation by some reasonable means, this is the first time you have received notice of violation of this License (for any work) from that copyright holder, and you cure the violation prior to 30 days after your receipt of the notice.

   Termination of your rights under this section does not terminate the licenses of parties who have received copies or rights from you under this License. If your rights have been terminated and not permanently reinstated, you do not qualify to receive new licenses for the same material under section 10.

9. Acceptance Not Required for Having Copies.

   You are not required to accept this License in order to receive or run a copy of the Program. Ancillary propagation of a covered work occurring solely as a consequence of using peer-to-peer transmission to receive a copy likewise does not require acceptance. However, nothing other than this License grants you permission to propagate or modify any covered work. These actions infringe copyright if you do not accept this License. Therefore, by modifying or propagating a covered work, you indicate your acceptance of this License to do so.

10. Automatic Licensing of Downstream Recipients.

    Each time you convey a covered work, the recipient automatically receives a license from the original licensors, to run, modify and propagate that work, subject

to this License. You are not responsible for enforcing compliance by third parties with this License.

An "entity transaction" is a transaction transferring control of an organization, or substantially all assets of one, or subdividing an organization, or merging organizations. If propagation of a covered work results from an entity transaction, each party to that transaction who receives a copy of the work also receives whatever licenses to the work the party's predecessor in interest had or could give under the previous paragraph, plus a right to possession of the Corresponding Source of the work from the predecessor in interest, if the predecessor has it or can get it with reasonable efforts.

You may not impose any further restrictions on the exercise of the rights granted or affirmed under this License. For example, you may not impose a license fee, royalty, or other charge for exercise of rights granted under this License, and you may not initiate litigation (including a cross-claim or counterclaim in a lawsuit) alleging that any patent claim is infringed by making, using, selling, offering for sale, or importing the Program or any portion of it.

11. Patents.

A "contributor" is a copyright holder who authorizes use under this License of the Program or a work on which the Program is based. The work thus licensed is called the contributor's "contributor version".

A contributor's "essential patent claims" are all patent claims owned or controlled by the contributor, whether already acquired or hereafter acquired, that would be infringed by some manner, permitted by this License, of making, using, or selling its contributor version, but do not include claims that would be infringed only as a consequence of further modification of the contributor version. For purposes of this definition, "control" includes the right to grant patent sublicenses in a manner consistent with the requirements of this License.

Each contributor grants you a non-exclusive, worldwide, royalty-free patent license under the contributor's essential patent claims, to make, use, sell, offer for sale, import and otherwise run, modify and propagate the contents of its contributor version.

In the following three paragraphs, a "patent license" is any express agreement or commitment, however denominated, not to enforce a patent (such as an express permission to practice a patent or covenant not to sue for patent infringement). To "grant" such a patent license to a party means to make such an agreement or commitment not to enforce a patent against the party.

If you convey a covered work, knowingly relying on a patent license, and the Corresponding Source of the work is not available for anyone to copy, free of charge and under the terms of this License, through a publicly available network server or other readily accessible means, then you must either (1) cause the Corresponding Source to be so available, or (2) arrange to deprive yourself of the benefit of the patent license for this particular work, or (3) arrange, in a manner consistent with the requirements of this License, to extend the patent license to downstream recipients. "Knowingly relying" means you have actual knowledge that, but for the patent license, your conveying the covered work in a country, or your recipient's use of the covered work in a country, would infringe one or more identifiable patents in that country that you have reason to believe are valid.

If, pursuant to or in connection with a single transaction or arrangement, you convey, or propagate by procuring conveyance of, a covered work, and grant a patent license to some of the parties receiving the covered work authorizing them to use, propagate, modify or convey a specific copy of the covered work, then the patent license you grant is automatically extended to all recipients of the covered work and works based on it.

A patent license is "discriminatory" if it does not include within the scope of its coverage, prohibits the exercise of, or is conditioned on the non-exercise of one or more of the rights that are specifically granted under this License. You may not convey a covered work if you are a party to an arrangement with a third party that is in the business of distributing software, under which you make payment to the third party based on the extent of your activity of conveying the work, and under which the third party grants, to any of the parties who would receive the covered work from you, a discriminatory patent license (a) in connection with copies of the covered work conveyed by you (or copies made from those copies), or (b) primarily for and in connection with specific products or compilations that contain the covered work, unless you entered into that arrangement, or that patent license was granted, prior to 28 March 2007.

Nothing in this License shall be construed as excluding or limiting any implied license or other defenses to infringement that may otherwise be available to you under applicable patent law.

12. No Surrender of Others' Freedom.

If conditions are imposed on you (whether by court order, agreement or otherwise) that contradict the conditions of this License, they do not excuse you from the conditions of this License. If you cannot convey a covered work so as to satisfy simultaneously your obligations under this License and any other pertinent obligations, then as a consequence you may not convey it at all. For example, if

you agree to terms that obligate you to collect a royalty for further conveying from those to whom you convey the Program, the only way you could satisfy both those terms and this License would be to refrain entirely from conveying the Program.

13. Use with the GNU Affero General Public License.

    Notwithstanding any other provision of this License, you have permission to link or combine any covered work with a work licensed under version 3 of the GNU Affero General Public License into a single combined work, and to convey the resulting work. The terms of this License will continue to apply to the part which is the covered work, but the special requirements of the GNU Affero General Public License, section 13, concerning interaction through a network will apply to the combination as such.

14. Revised Versions of this License.

    The Free Software Foundation may publish revised and/or new versions of the GNU General Public License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns.

    Each version is given a distinguishing version number. If the Program specifies that a certain numbered version of the GNU General Public License "or any later version" applies to it, you have the option of following the terms and conditions either of that numbered version or of any later version published by the Free Software Foundation. If the Program does not specify a version number of the GNU General Public License, you may choose any version ever published by the Free Software Foundation.

    If the Program specifies that a proxy can decide which future versions of the GNU General Public License can be used, that proxy's public statement of acceptance of a version permanently authorizes you to choose that version for the Program.

    Later license versions may give you additional or different permissions. However, no additional obligations are imposed on any author or copyright holder as a result of your choosing to follow a later version.

15. Disclaimer of Warranty.

    THERE IS NO WARRANTY FOR THE PROGRAM, TO THE EXTENT PERMITTED BY APPLICABLE LAW. EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE PROGRAM "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF

MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE PROGRAM IS WITH YOU. SHOULD THE PROGRAM PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

16. Limitation of Liability.

    IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MODIFIES AND/OR CONVEYS THE PROGRAM AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE PROGRAM (INCLUDING BUT NOT LIMITED TO LOSS OF DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD PARTIES OR A FAILURE OF THE PROGRAM TO OPERATE WITH ANY OTHER PROGRAMS), EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

17. Interpretation of Sections 15 and 16.

    If the disclaimer of warranty and limitation of liability provided above cannot be given local legal effect according to their terms, reviewing courts shall apply local law that most closely approximates an absolute waiver of all civil liability in connection with the Program, unless a warranty or assumption of liability accompanies a copy of the Program in return for a fee.

End of Terms and Conditions

How to Apply These Terms to Your New Programs

If you develop a new program, and you want it to be of the greatest possible use to the public, the best way to achieve this is to make it free software which everyone can redistribute and change under these terms.

To do so, attach the following notices to the program. It is safest to attach them to the start of each source file to most effectively state the exclusion of warranty; and each file should have at least the "copyright" line and a pointer to where the full notice is found.

> <one line to give the program's name and a brief idea of what it does.>
>
> Copyright (C) <year> <name of author>
>
> This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the

> Free Software Foundation, either version 3 of the License, or (at your option) any later version.
>
> This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.
>
> You should have received a copy of the GNU General Public License along with this program. If not, see http://www.gnu.org/licenses/.

Also add information on how to contact you by electronic and paper mail.

If the program does terminal interaction, make it output a short notice like this when it starts in an interactive mode:

> <program> Copyright (C) <year> <name of author>
>
> This program comes with ABSOLUTELY NO WARRANTY; for details type `show w`. This is free software, and you are welcome to redistribute it under certain conditions; type `show c` for details.

The hypothetical commands `show w` and `show c` should show the appropriate parts of the General Public License. Of course, your program's commands might be different; for a GUI interface, you would use an "about box".

You should also get your employer (if you work as a programmer) or school, if any, to sign a "copyright disclaimer" for the program, if necessary. For more information on this, and how to apply and follow the GNU GPL, see http://www.gnu.org/licenses/.

The GNU General Public License does not permit incorporating your program into proprietary programs. If your program is a subroutine library, you may consider it more useful to permit linking proprietary applications with the library. If this is what you want to do, use the GNU Lesser General Public License instead of this License. But first, please read http://www.gnu.org/philosophy/why-not-lgpl.html.

Appendix C.

ALGORITHMS AS INFERENCE RULES

There are a number of ways to express algorithms in an article of computer science. Some traditional way writes imperative pseudo code that emphasizes state changing (assignments) and loops, as shown in Algorithm 3.1. While pseudo code in this style can be straightforwardly translated to a common imperative programming language, such as C++, it is less mathematically formal and rigorous, with a big burden to reason about. On the other hand, functional style algorithms use equations to introduce new variables and bindings, without changing the value bound to a variable. It is obvious that updating the state of a variable is only useful when running in a loop. The functional style employs recursive calls to achieve much more powerful repetitions and bind new values to variables in inner calls. It is a more concise and rigorous way to express algorithms, and a better form to reason about the correctness and properties.

Inference rules Functional style algorithms often use pattern matching with guards to perform case analyses on the input arguments, and bind new variables to components and intermediate results for further specification. Just like the chained conditional statements in traditional pseudo code, these cases are listed alongside the left for clarity. This is space wasting for publishing articles in journals and proceedings. An elegant space saving way of listing the possibly nested cases is to formulate them as a sequence of *inference rules*,

$$\text{(Case 1)}\ \frac{A \quad B \quad C}{\text{(Inner Case)}\ \dfrac{F \quad G}{K_1} \qquad \text{(Inner Else)}\ \dfrac{}{K_2}} \qquad \text{(Case 2)}\ \frac{D \quad E}{K_3} \qquad \text{(Else)}\ \frac{}{K_4},$$

where A, B, C, D, E are the top-level premises, F, G the inner-level premises of the rule labeled (Inner Case), K_1, K_2, K_3, K_4 the conclusions. Inference rules are examined sequentially, thus, $\neg(A \wedge B \wedge C)$ is implied in (Case 2). For example, the Euclid's greatest common divisor algorithm $gcd(m, n)$ can be defined on integers m, n as follows,

$$gcd(m, n) = \quad \frac{m < 0}{gcd(-m, n)} \quad \frac{n < 0}{gcd(m, -n)} \quad \frac{m < n}{gcd(n, m)} \quad \frac{n = 0}{m} \quad \frac{}{gcd(n, m - n)}.$$

Generally, functional style algorithms are more abstract and less to write. However, for accumulation over a collection of elements, some verbose arrangement is necessary

to explicitly handle the iteration of elements. Let's see another example to translate dfs_match in Algorithm 3.1 to inference rules.

$$\mathsf{dfs_match}'(f, \emptyset, E, E') = f,$$

$$\mathsf{dfs_match}'(f, [(p, p')] \uplus P, E, E') = \frac{p \in \mathrm{dom}(f)}{\dfrac{f(p) \neq p'}{\textbf{raise } \textit{Mismatch}} \quad \dfrac{}{\mathsf{dfs_match}'(f, P, E, E')}}$$

$$\frac{\tau(p') \not\prec \tau(p)}{\textbf{raise } \textit{Mismatch}} \qquad \frac{p' \in \mathrm{ran}(f)}{\textbf{raise } \textit{Mismatch}} \qquad \frac{\exists p \xrightarrow{a} q \in E \bullet p' \xrightarrow{a} q' \notin E'}{\textbf{raise } \textit{Mismatch}}$$

$$\frac{Q = [(q, q') \mid p \xrightarrow{a} q \in E \wedge p' \xrightarrow{a} q' \in E']}{\mathsf{dfs_match}'(f \cup \{p \mapsto p'\}, Q \uplus P, E, E')}.$$

The use of list concatenation $\uplus$ in the above definition implements the stack pop and push that guarantee the DFS.

Overlined terms To reduce the occasions of explicit enumeration of elements over a collection, *overlined terms* are often used in certain cases. To the simplest case, an overlined varible $\overline{v}$ is expanded to a series of variables $v_1, \ldots, v_n$. This expansion is usually restricted by the context, where a component may already be defined. A variable that is defined as a singlton repeats itself in the expansion. A variable that is also a series pairs with the other series element-wise in the expansion. For example, let f be a defined function, $\overline{a}$ a series. Then $\overline{b = 2a}$ expands to $b_1 = 2a_1, \ldots, b_n = 2a_n$, and $\overline{f(a, b)}$ expands to $f(a_1, b_1), \ldots, f(a_n, b_n)$. With the overlined terms, dfs_match′ can be re-written without the list comprehension.

$$\mathsf{dfs_match}'(f, [(p, p')] \uplus P, E, E') = \qquad (\textsc{visited})\ \frac{f(p) = p'}{\mathsf{dfs_match}'(f, P, E, E')}$$

$$(\textsc{explore})\ \frac{\tau(p') \prec \tau(p) \quad \overline{p \xrightarrow{a} q \in E \implies p' \xrightarrow{a} q' \in E'} \quad p \notin \mathrm{dom}(f) \quad p' \notin \mathrm{ran}(f)}{\mathsf{dfs_match}'(f \cup \{p \mapsto p'\}, [\overline{(q, q')}] \uplus P, E, E')} \qquad \frac{}{\textbf{raise } \textit{Mismatch}}.$$

I would say the overline used in this definition is quite intuitive. If a detailed explanation must be given, p, p', E, E' are bound variable in the context, a, q, q' are new series. Since q and q' are all aligned with a, they have the correspondence synchronized by a. Therefore, we can expand the overlined term as, for all the outgoing edge labels $a_1, \ldots, a_n$,

$$p \xrightarrow{a_1} q_1 \in E \implies p' \xrightarrow{a_1} q_1' \in E', \quad \ldots, \quad p \xrightarrow{a_n} q_n \in E \implies p' \xrightarrow{a_n} q_n' \in E',$$

and we have the list of pairs $[(q_1, q_1'), \ldots, (q_n, q_n')]$ in the conclusion.

I must admit that the rules for the expansion are not rigorous. There are some ambiguities that require common sense to remove. What I really mean in the above definition is

$$\forall p \xrightarrow{a} q \in E \bullet p' \xrightarrow{a} q' \in E',$$

but it cannot be written as a simple series of prepositions, without specifying the range of a in a verbose way. I don't want to sacrifice the simplicity for the negligible bit of additional rigor. At the end of the day, we can still complement the specification with remarks in the text.

A formally available construct for expandable series is the *variadic templates* in C++ [31]. It provides some great ideas for expanding series and pairing variables. An exciting feature of this construct is the *fold expression* for iteration and accumulation of elements in a series.

```
template<class... T>
void print(T... a)
{
    (std::cout << ... << a) << std::endl;
    ((std::cout << '␣' << a), ...) << std::endl;
}

int main()
{
    print('x', 100, true, 99.87);
    return 0;
}
```

Appendix D.

BIOGRAPHY


0
cm
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24